Ben: API KEYLER KULLANARAK KOD YAZAN AJANLAR YAPACAĞIZ AMA HER BİR MODÜLÜ AYRI AJANLAR YAPACAK VE UYUMLU OLMALI

GPT:Tamam, sade ve direkt veriyorum:

🧠 SİSTEM

```
Orchestrator
 ├── Planner
 ├── CodeGenerator
 ├── Reviewer
 ├── Integration
 ├── Test
 └── Deploy
```

⚙️ AJANLAR

1. Planner

```
{
  "module": "auth_service",
  "input": ["email", "password"],
  "output": ["jwt_token"],
  "dependencies": ["database"]
}
```

---

2. Code Generator

Kurallar:

Tek framework (FastAPI)

Tek stil (PEP8)

Sabit yapı

---

3. Reviewer

Kod kontrol

Standart dışı → RED

---

4. Integration

interface.json okur

modülleri bağlar

---

5. Test

unit test

integration test

---

6. Deploy

docker build

deploy

🔑 API KEY

Agent → KeyManager → API

```
def call_api(prompt):
    key = get_key("openai")
    return request(key, prompt)
```

🧱 MODÜL YAPISI

```
module/
 ├── main.py
 ├── interface.json
 ├── requirements.txt
 └── tests/
```

```
{
  "endpoint": "/login",
  "method": "POST",
  "input": {
    "email": "string",
    "password": "string"
  },
  "output": {
    "token": "string"
  }
}
```

🧬 İLETİŞİM

Sadece JSON:

```
{
  "action": "generate_code",
  "module": "auth_service"
}
```

⚡ STACK

Python + FastAPI

LangGraph / CrewAI

Redis / RabbitMQ

PostgreSQL

Docker

🚨 KURAL

Serbest ajan YOK

Farklı teknoloji YOK

Direkt bağlantı YOK

Ben:Bir uygulamanin en diş katmanlari nedir fronend backend veritabanı gibi ? Daha var mi

GPT:En dış katmanları sade şekilde şöyle düşünebilirsin:

🧱 TEMEL KATMANLAR

1. Frontend (İstemci)

Kullanıcının gördüğü yer

Web / mobil / desktop

Örnek: React, Flutter

---

2. Backend (Sunucu)

İş mantığı burada

API üretir

Frontend ile veritabanı arasında köprü

---

3. Veritabanı (Database)

Veriler burada tutulur

Örnek: PostgreSQL, MongoDB

➕ DAHA VAR (GERÇEK SİSTEMDE)

4. API Gateway

Tüm istekler buradan geçer

Auth, rate limit vs.

---

5. Authentication / Authorization

Giriş sistemi

Yetki kontrolü

---

6. Cache

Hız için geçici veri

Örnek: Redis

---

7. Message Queue

Arka plan işlemleri

Örnek: RabbitMQ

---

8. Storage (Dosya sistemi)

Resim, video vs.

Örnek: S3

---

9. CDN

Statik dosyaları hızlı dağıtır

---

10. DevOps / Deployment

Docker, CI/CD

Sunucu yönetimi

---

11. Monitoring / Logging

Hata takibi

Performans izleme

🧠 KISA ÖZET

User
↓
Frontend
↓
API Gateway
↓
Backend
↓
Cache / Queue
↓
Database + Storage

Ben:daha fazla detay

GPT:Evet, daha da var. Tam ve geniş listeyi en dıştan içe + yan katmanlarla veriyorum:

🌍 GENİŞLETİLMİŞ TAM KATMAN LİSTESİ

1. Kullanıcı (User)

2. Cihaz / İşletim Sistemi

3. Tarayıcı / Mobil Runtime

4. DNS

5. CDN

6. Edge Computing (Cloudflare Workers vb.)

7. WAF

8. DDoS Protection

9. Load Balancer

10. Reverse Proxy (Nginx vb.)

11. API Gateway

12. Authentication (AuthN)

13. Authorization (AuthZ)

14. Session Management

15. Frontend (UI)

16. BFF (Backend for Frontend)

17. Backend (Application Layer)

18. Microservices / Service Layer

19. Business Logic Layer

20. API Layer (REST / GraphQL)

21. Rate Limiting

22. Input Validation / Sanitization

23. Cache (Redis)

24. Message Queue (RabbitMQ / Kafka)

25. Event Streaming

26. Database (SQL / NoSQL)

27. Read Replicas

28. Data Warehouse

29. Data Lake

30. Object Storage (S3)

31. File Processing Service

32. Search Engine (Elasticsearch)

33. Logging

34. Monitoring

35. Alerting

36. Tracing (Distributed Tracing)

37. Analytics

38. Telemetry

39. CI/CD Pipeline

40. Artifact Registry

41. Container Runtime (Docker)

42. Orchestration (Kubernetes)

43. Service Discovery

44. Configuration Management

45. Secrets Management

46. Infrastructure (Cloud / VPS)

47. Network Layer (VPC, Subnet)

48. Firewall (Network level)

49. Backup System

50. Disaster Recovery

🧠 KISA GERÇEK

Bunun da ötesi var SONRA DÜŞÜNÜRÜZ UNUTMA (fintech, AI pipeline vs.)

Ben:Bubların hepsini ayrı ajanlar yapsa daha mı iyidir?

Ama iş gerçekten büyük,biz uygulama yazan uygulama yazacağız ama tek ajans verirsek basit iş yapar hepsini detaylı yapmalı

Dediğin şey doğru ama çözüm “her şeyi ayrı ajan” değil.

👉 Büyük sistem = çok ajan
👉 Ama doğru olan = hiyerarşik ajan sistemi (katmanlı kontrol)

🔥 DOĞRU MODEL: HİYERARŞİK AJAN SİSTEMİ

Tek level değil, katmanlı ajanlar:

```
👑 Master Orchestrator
 ├── 🧠 Domain Orchestrators
 │    ├── Frontend Orchestrator
 │    ├── Backend Orchestrator
 │    ├── Data Orchestrator
 │    ├── Infra Orchestrator
 │    └── Security Orchestrator
 │
 │    ├── (alt ajanlar)
 │    │    ├── Code Agents
 │    │    ├── Config Agents
 │    │    ├── Test Agents
 │    │    └── Validator Agents
```

🧠 MANTIK

Senin düşündüğün:

> “çok ajan = daha iyi”

Doğrusu:

> “kontrollü çok ajan = daha iyi”

⚙️ NASIL BÖLECEKSİN

1. ÜST SEVİYE (Orchestrator)

sistemi yönetir

görev dağıtır

kuralları zorlar

---

2. DOMAIN AJANLARI

Frontend Orchestrator

UI component agent

state management agent

styling agent

Backend Orchestrator

API agent

business logic agent

auth agent

Data Orchestrator

schema agent

query agent

migration agent

Infra Orchestrator

docker agent

kubernetes agent

network agent

Security Orchestrator

auth policy agent

validation agent

vulnerability scanner agent

🧱 ALT AJANLAR (GERÇEK ÜRETİM)

Burada istediğin kadar parçalayabilirsin:

Örnek:

```
Auth Module
 ├── Login Agent
 ├── Token Agent
 ├── Encryption Agent
 ├── Validation Agent
```

🚨 KRİTİK KURAL

👉 Ajanlar rastgele değil,
👉 üst ajan tarafından kontrol edilerek çalışmalı

⚡ NEDEN BU DAHA GÜÇLÜ

Detaylı üretim var ✔

Kaos yok ✔

Standart korunur ✔

Ölçeklenir ✔

💣 GERÇEK OLAY

Büyük şirketler şunu yapar:

microservices ✔

ama üstünde orchestration ✔

Senin sistem: 👉 AI ile otomatik microservice fabrikası

🧠 SON NET

✔ Çok ajan → EVET
❌ Kontrolsüz çok ajan → HAYIR
✔ Katmanlı + orchestrator → DOĞRU

Dahada geliştir ajan sistemini çünkü ben groq API ye yaptırdığım için çakma işle yaptırıyor her birini detaylı yaptıracak bir algoritma kurdurmaliyim,promt ile değil algoritma ile çalişsinlar.

Aynen doğru noktaya bastın:
👉 “prompt yaz → iyi sonuç gelsin” yaklaşımı yetersiz
👉 Senin ihtiyacın: zorlayan, doğrulayan, iteratif bir üretim algoritması

Bunu sana net, uygulanabilir şekilde kuruyorum:

🔥 TEMEL FİKİR

Ajanlar tek seferde üretmez
➡️ üret → kontrol → düzelt → tekrar üret döngüsü

🧠 ÇEKİRDEK ALGORİTMA

```
while not DONE:
    plan = Planner()
    code = Generator(plan)
    review = Reviewer(code)

    if review.fail:
        code = Fixer(review.feedback)
    else:
        test = Tester(code)

        if test.fail:
            code = Fixer(test.feedback)
        else:
            DONE = True
```

⚙️ AJANLARI YÜKSELT (SIRADAN AJAN DEĞİL)

1. Planner → “Detay zorlayıcı”

Sadece modül ismi üretmesin, şunu zorla:

```
{
  "module": "auth",
  "edge_cases": [
    "invalid email",
    "expired token",
    "sql injection"
  ],
  "performance_requirements": {
    "max_response_ms": 200
  },
  "security_requirements": [
    "hash_password",
```

```
    "jwt_expiry"
  ]
}
```

👉 Eğer bunlar yoksa → RED

---

2. Generator → “Kısıtlı üretici”

allowed libraries listesi ver

forbidden listesi ver

max complexity ver

👉 kontrolsüz üretim engellenir

---

3. Reviewer → “acımasız denetçi”

Sadece “iyi mi?” demesin:

```
{
  "score": 65,
  "fail_reasons": [
    "no error handling",
    "no validation",
    "security weak"
  ],
  "must_fix": true
}
```

👉 score < 85 → RED

---

4. Tester → “otomatik zorlayıcı”

unit test üretir

edge-case test üretir

load test simülasyonu

👉 test geçmeden ilerleme YOK

---

5. Fixer → “hedefli düzeltici”

Rastgele düzeltme yok:

```
{
  "fix_targets": [
    "add input validation",
    "handle null values"
  ]
}
```

💣 KRİTİK MEKANİZMA: SELF-CRITIC LOOP

Her ajan kendini de sorgular:

```
result = agent()
critique = self_critic(result)

if critique.fail:
    retry()
```

🧬 2. SEVİYE: ÇAPRAZ DENETİM (EN GÜÇLÜ SİSTEM)

Ajanlar birbirini denetler:

Generator → code üretir
Reviewer → eleştirir
Security Agent → ayrı kontrol eder
Performance Agent → ayrı kontrol eder

👉 tek reviewer YETERSİZ

⚡ 3. ZORLAMA SİSTEMİ (PROMPT DEĞİL, KURAL)

Senin sistemde:

```
IF:
 - test yoksa
 - validation yoksa
 - error handling yoksa
THEN:
 → reject + retry
```

🧠 4. STATE MACHINE (ÇOK ÖNEMLİ)

Her modül state ile ilerler:

```
PLANNED
→ GENERATED
→ REVIEWED
→ TESTED
→ APPROVED
→ INTEGRATED
```

👉 state atlaması YASAK

🧱 5. HARD CONSTRAINT ENGINE

Kurallar kodda olmalı:

```
if "try:" not in code:
    reject("no error handling")

if "validate" not in code:
    reject("no validation")
```

🚨 6. GROQ PROBLEMİNİ ÇÖZME

Groq hızlı ama yüzeysel çalışır. Çözüm:

👉 “tek büyük output” isteme
👉 “küçük parçalar + sürekli kontrol” yap

🧠 SON MODEL

```
Orchestrator
 ├── Planner
 ├── Generator
 ├── Reviewer
 ├── Security Checker
 ├── Performance Checker
 ├── Tester
 ├── Fixer
 └── State Manager
```

💥 SONUÇ

Senin sistem:

👉 prompt değil
👉 zorlayan üretim makinesi

İstersen sırada:

👉 sana gerçek Python orchestrator kodu yazayım
(ajanlar gerçekten loop içinde çalışsın)

veya

👉 “auth sistemi ajanlarla üretelim” (canlı simülasyon)

hangisini istiyorsun?

Ben:Tamam şimdi bu tasarlanması gereken ajanlar neler: Tamam, sade ve net: tasarlanması gereken ajanlar (hiyerarşik + görev bazlı)

🧠 1. CORE (ZORUNLU)

1. Orchestrator Agent

2. State Manager Agent

3. Task Distributor Agent

🧩 2. PLANLAMA

4. Planner Agent

5. Architecture Agent

6. Dependency Agent

🏗️ 3. ÜRETİM

Backend

7. API Agent

8. Business Logic Agent

9. Auth Agent

Frontend

10. UI Component Agent

11. State Management Agent

12. Styling Agent

Data

13. Schema Agent

14. Query Agent

15. Migration Agent

🔐 4. GÜVENLİK

16. Validation Agent

17. Security Agent

18. Encryption Agent

⚙️ 5. ALTYAPI

19. Docker Agent

20. Kubernetes Agent

21. Network Agent

22. Config Agent

23. Secrets Manager Agent

🧪 6. TEST & KALİTE

24. Unit Test Agent

25. Integration Test Agent

26. Performance Test Agent

27. Code Reviewer Agent

28. Linter Agent

🔍 7. KONTROL & OPTİMİZASYON

29. Performance Optimizer Agent

30. Refactor Agent

31. Complexity Analyzer Agent

🔗 8. ENTEGRASYON

32. Integration Agent

33. API Contract Agent

📊 9. GÖZLEMLEME

34. Logging Agent

35. Monitoring Agent

36. Alerting Agent

🚀 10. DAĞITIM

37. CI/CD Agent

38. Deployment Agent

🧬 11. GELİŞMİŞ (OPSİYONEL AMA GÜÇLÜ)

39. Self-Critic Agent

40. Fixer Agent

41. Audit Agent

42. Documentation Agent

💣 NET SONUÇ

Toplam: ~40 ajan

Ama:

👉 Hepsi aynı seviyede değil
👉 Gruplar halinde çalışmalı
👉 Orchestrator kontrol etmeli

İstersen sonraki adım:

👉 bu 40 ajanı gerçek çalışan sistemde nasıl bağlayacağını çizeyim
👉 veya direkt kod mimarisi (dosya yapısı + agent class yapısı) kurayım

İşte asıl kritik nokta bu 👇
👉 Ajanlar kullanıcının ne istediğini tahmin etmez
👉 tanımlı, zorunlu bir “spec (şartname)” üzerinden çalışır

Yani çözüm:
“prompt değil → yapılandırılmış gereksinim sistemi”

🔥 1. TEMEL: REQUIREMENT ENGINE

Kullanıcıdan serbest metin alma → YETERSİZ
Bunun yerine:

```
{
  "app_name": "Todo App",
  "features": [
```

```
    "user login",
    "task create",
    "task delete"
  ],
  "constraints": {
    "max_response_ms": 200,
    "security": "high"
  },
  "platform": "web"
}
```

👉 Bu olmadan ajanlar çalışmaz

🧠 2. REQUIREMENT PARSER AGENT

Kullanıcı ne yazarsa yazsın, bunu parçalar:

Input:

> “login sistemi olan hızlı bir app yap”

Output:

```
{
  "modules": ["auth"],
  "requirements": {
    "auth": {
      "methods": ["jwt"],
      "validation": true,
      "security_level": "high"
    }
  }
}
```

⚙️ 3. SPEC GENERATOR (EN KRİTİK)

Bu ajan sistemi ZORLAR:

```
{
  "auth_module": {
```

```
    "input": ["email", "password"],
    "output": ["token"],
    "rules": [
      "password hashed",
      "email validated",
      "jwt expiry 1h"
    ],
    "edge_cases": [
      "invalid email",
      "wrong password",
      "expired token"
    ]
  }
}
```

👉 Artık ajanlar “ne yapacağını bilir”

🧱 4. CONTRACT SYSTEM (ZORUNLU)

Her modül şu contract ile çalışır:

```
{
  "endpoint": "/login",
  "method": "POST",
  "input_schema": {...},
  "output_schema": {...}
}
```

👉 Generator buna UYMAK ZORUNDA

🔍 5. EXPECTATION CHECKER

Kullanıcı isteği vs üretilen sistem karşılaştırılır:

```
{
  "missing": ["password reset"],
  "extra": ["social login"],
  "score": 82
}
```

👉 score düşük → sistem RED

🧬 6. FEEDBACK LOOP

User → Requirement → Spec → Code → Test → Compare → Fix → Repeat

---

💣 7. ZORLAMA KURALI

Ajanlar özgür değil:

```
if spec_not_complete:
    reject()

if code_not_matching_spec:
    reject()

if feature_missing:
    reject()
```

⚡ 8. GERÇEK AKIŞ

1. User input
2. Requirement Parser
3. Spec Generator
4. Planner
5. Generator
6. Reviewer
7. Tester
8. Expectation Checker
9. Fixer

🧠 NET CEVAP

Ajanlar kullanıcıyı anlamaz
👉 Onlara anlatılmış, zorunlu spec verilir

🔥 ÖZET

Prompt = giriş

Spec = gerçek gerçeklik

Ajanlar = spec executor

İstersen:

👉 sana otomatik spec üreten sistemin kodunu yazayım
👉 ya da “kullanıcı yazıyor → sistem full app çıkarıyor” pipeline’ı kurayım

```
📱 Sunum Katmanı
   ├── UI Görseller / Ekranlar
   └── UI Mantığı / Ekran Kontrolcüleri (ViewModel vb.)
🤖 Uygulama / İş Mantığı Katmanı
   ├── İş Kuralları
   ├── İş Akışları
   └── Orkestrasyon
💾 Veri Erişim Katmanı
   ├── Repository / DAO
   └── API / Network Access
🔐 Veri / Depolama Katmanı
   ├── Veritabanı
   ├── Dosya Depolama
   └── Cache / Harici Kaynaklar
```

```
📌 Uygulama Mimarisi
│
├── A) Katmanlı / N-Tier Mimari (Layered Architecture)
│   ├─ 1) Sunum / UI Katmanı (Presentation)
│   │   ├── UI Görseller / Ekranlar
│   │   ├── UI Mantığı / UI Logic
│   │   └── Çoklu istemci (Web, mobil, API, CLI)
│   │
│   ├─ 2) Uygulama Katmanı (Application)
│   │   ├── İş akışları / use case’ler
│   │   ├── API endpoint’ler / controllers
│   │   └── Servis koordinasyonu
│   │
│   ├─ 3) İş / Domain Katmanı (Business)
```

```
│   │   ├── İş kuralları
│   │   ├── Domain modelleri
│   │   ├── Servisler / domain logic
│   │   └── Kurallar / validation
│   │
│   ├─ 4) Veri Servis / Persistence Katmanı
│   │   ├── Repository / DAO
│   │   ├── ORM / query logic
│   │   └── Cache erişimi
│   │
│   └─ 5) Veri / Database / Storage
│       ├── Veritabanı (SQL, NoSQL)
│       ├── Dosya / blok depolama
│       └── Data lakes / dış API veri
|
├── B) Clean / Onion / Hexagonal Variants
│   ├─ 1) Entities / Core Domain
│   ├─ 2) Use Cases / Application Logic
│   ├─ 3) Ports / Interfaces
│   ├─ 4) Adapters / UI & DB
│   └─ 5) External Interfaces & Services
|
├── C) Microservices / Distributed Services
│   ├─ 1) Microservice A (checkout)
│   ├─ 2) Microservice B (ürün kataloğu)
│   ├─ 3) Microservice C (ödeme)
│   ├─ API Gateway
│   └─ Event / Message Bus
|
├── D) Client-Server
│   ├─ 1) Client
│   │   ├── Web UI
│   │   └── Mobil UI
│   └─ 2) Server
│       ├── API
│       └── DB
|
├── E) Modern / SaaS & Cloud-Native
│   ├─ 1) Frontend (SPA / Headless)
│   ├─ 2) API Layer
│   ├─ 3) Business Logic Microservices
│   ├─ 4) Cloud Storage / DB
│   └─ 5) Platform Services
|
└── F) UI-Centric Patterns
    ├─ MVC (Model-View-Controller)
    │   ├── Model
    │   ├── View
```

```
│   └── Controller
│
├── MVVM (Model-View-ViewModel)
│   ├── View
│   ├── ViewModel
│   └── Model
│
└── MVP (Model-View-Presenter)
    ├── Model
    ├── View
    └── Presenter
```

📌 Uygulama Mimarisi – A'dan Z'ye Derin Ağaç 🧠

```
│
├── 1) Sunum / Presentation Layer
│   ├── UI Görsel Bileşenler
│   │   ├── UI Kit / Framework (React, Vue, Flutter, SwiftUI vb.)
│   │   ├── Bileşenler / Widgets
│   │   └── Ekranlar / Sayfalar
│   │
│   ├── UI Mantığı
│   │   ├── View Controllers / Presenters
│   │   ├── ViewModels / States
│   │   ├── Event Handlers (Klik, Kaydır, Yazı Girişi)
│   │   └── UI Validation / Error Handling
│   │
│   ├── Sunum Desenleri
│   │   ├── MVC (Model-View-Controller)
│   │   ├── MVVM (Model-View-ViewModel)
│   │   └── MVP (Model-View-Presenter)
│   │
│   └── İstemci Çeşitleri
│       ├── Web UI
│       ├── Mobil UI
│       ├── CLI (Komut Satırı)
│       └── API Gateway (Kullanıcıya açık API yüzü)
│
├── 2) Uygulama / Application Layer
│   ├── API Katmanı
│   │   ├── REST Endpoints
│   │   ├── GraphQL
│   │   └── gRPC
│   │
│   ├── API Controller / Handler
│   │   ├── Request Validation
│   │   ├── Routing
```

```
│   │   └── Auth Middleware
│   │
│   ├── Uygulama Servisleri
│   │   ├── Use Cases
│   │   ├── Application Orchestration
│   │   └── DTO / Mapper'lar
│   │
│   └── Teknik Alt Yapı
│       ├── Middleware
│       ├── Logging / Audit
│       └── Caching Interface
│
├── 3) İş / Domain Katmanı
│   ├── Domain Modelleri
│   │   ├── Entity / Value Object
│   │   └── Aggregate
│   │
│   ├── İş Servisleri
│   │   ├── Domain Services
│   │   ├── Factories
│   │   └── Policy / Rules
│   │
│   ├── Domain Event'ler
│   │   ├── Event Publishing
│   │   ├── Event Handlers
│   │   └── Integration Events
│   │
│   └── DDD Pattern'leri
│       ├── Bounded Context
│       ├── Repository Interface
│       └── CQRS / Command-Query
│
├── 4) Veri Erişim / Persistence Layer
│   ├── Repository & DAO
│   │   ├── CRUD Methods
│   │   └── Query Builders
│   │
│   ├── ORM / SDK
│   │   ├── Entity Framework
│   │   ├── Hibernate
│   │   └── Sequelize / Prisma
│   │
│   ├── Cache / Temp Store
│   │   ├── Redis
│   │   └── In-Memory
│   │
│   └── Data Abstraction
│       ├── Interfaces
```

```
│       └── Adapter Pattern
│
├── 5) Veri / Database / Storage
│   ├── SQL Veritabanı
│   │   ├── Tablo, İlişki
│   │   ├── Index, View
│   │   └── Stored Procedures
│   │
│   ├── NoSQL
│   │   ├── Document DB
│   │   ├── Key-Value
│   │   └── Graph DB
│   │
│   ├── Blob / File Storage
│   │   ├── Cloud Storage
│   │   └── File System
│   │
│   └── Data Lake / Warehouse
│
├── 6) Servis & Entegrasyon Katmanı
│   ├── Microservices
│   │   ├── Bounded Context bazlı servis
│   │   ├── Event Bus
│   │   └── Service Discovery
│   │
│   ├── API Gateway
│   ├── External API Integrations
│   │   ├── HTTP Clients
│   │   └── Authentication
│   │
│   └── Messaging
│       ├── Kafka / RabbitMQ
│       └── Pub/Sub
│
├── 7) Infra / DevOps
│   ├── Deployment
│   │   ├── Container / Orchestrator
│   │   └── CI/CD
│   │
│   ├── Monitoring
│   │   ├── Logs
│   │   └── Metrics
│   │
│   └── Security
│       ├── Auth/Zauth
│       └── Secret Management
│
└── 8) Pattern / Stil Katmanları
```

```
    ├── Layered (N-Tier) – Tipik üç katman: UI, Logic, Data 1
    ├── Onion / Hexagonal – Domain merkezli herkes adapter üzerinden konuşur 2
    ├── Microkernel / Plugin
    ├── Event-Driven
    ├── CQRS / Event Sourcing
    └── Serverless / Cloud Native

📌 Uygulama Mimarisi
│
├── A) Katmanlı / N-Tier Mimari (Layered Architecture)
│   ├─ 1) Sunum / UI Katmanı (Presentation)
│   │   ├── UI Görseller / Ekranlar
│   │   ├── UI Mantığı / UI Logic
│   │   └── Çoklu istemci (Web, mobil, API, CLI)
│   │
│   ├─ 2) Uygulama Katmanı (Application)
│   │   ├── İş akışları / use case'ler
│   │   ├── API endpoint'ler / controllers
│   │   └── Servis koordinasyonu
│   │
│   ├─ 3) İş / Domain Katmanı (Business)
│   │   ├── İş kuralları
│   │   ├── Domain modelleri
│   │   ├── Servisler / domain logic
│   │   └── Kurallar / validation
│   │
│   ├─ 4) Veri Servis / Persistence Katmanı
│   │   ├── Repository / DAO
│   │   ├── ORM / query logic
│   │   └── Cache erişimi
│   │
│   └─ 5) Veri / Database / Storage
│       ├── Veritabanı (SQL, NoSQL)
│       ├── Dosya / blok depolama
│       └── Data lakes / dış API veri
│
├── B) Clean / Onion / Hexagonal Variants
│   ├─ 1) Entities / Core Domain
│   ├─ 2) Use Cases / Application Logic
│   ├─ 3) Ports / Interfaces
│   ├─ 4) Adapters / UI & DB
│   └─ 5) External Interfaces & Services
│
├── C) Microservices / Distributed Services
│   ├─ 1) Microservice A (checkout)
│   ├─ 2) Microservice B (ürün kataloğu)
│   ├─ 3) Microservice C (ödeme)
│   ├─ API Gateway
```

```
│   └─ Event / Message Bus
│
├── D) Client-Server
│   ├─ 1) Client
│   │   ├── Web UI
│   │   └── Mobil UI
│   └─ 2) Server
│       ├── API
│       └── DB
│
├── E) Modern / SaaS & Cloud-Native
│   ├─ 1) Frontend (SPA / Headless)
│   ├─ 2) API Layer
│   ├─ 3) Business Logic Microservices
│   ├─ 4) Cloud Storage / DB
│   └─ 5) Platform Services
│
└── F) UI-Centric Patterns
├─ MVC (Model-View-Controller)
│   ├── Model
│   ├── View
│   └── Controller
│
├─ MVVM (Model-View-ViewModel)
│   ├── View
│   ├── ViewModel
│   └── Model
│
└─ MVP (Model-View-Presenter)
├── Model
├── View
└── Presenter
```

📌 Uygulama Mimarisi – A'dan Z'ye Derin Ağaç 🧠

```
│
├── 1) Sunum / Presentation Layer
│   ├── UI Görsel Bileşenler
│   │   ├── UI Kit / Framework (React, Vue, Flutter, SwiftUI vb.)
│   │   ├── Bileşenler / Widgets
│   │   └── Ekranlar / Sayfalar
│   │
│   ├── UI Mantığı
│   │   ├── View Controllers / Presenters
│   │   ├── ViewModels / States
│   │   ├── Event Handlers (Klik, Kaydır, Yazı Girişi)
│   │   └── UI Validation / Error Handling
│   │
│   ├── Sunum Desenleri
```

```
│   │   ├── MVC (Model-View-Controller)
│   │   ├── MVVM (Model-View-ViewModel)
│   │   └── MVP (Model-View-Presenter)
│   │
│   └── İstemci Çeşitleri
│       ├── Web UI
│       ├── Mobil UI
│       ├── CLI (Komut Satırı)
│       └── API Gateway (Kullanıcıya açık API yüzü)
│
├── 2) Uygulama / Application Layer
│   ├── API Katmanı
│   │   ├── REST Endpoints
│   │   ├── GraphQL
│   │   └── gRPC
│   │
│   ├── API Controller / Handler
│   │   ├── Request Validation
│   │   ├── Routing
│   │   └── Auth Middleware
│   │
│   ├── Uygulama Servisleri
│   │   ├── Use Cases
│   │   ├── Application Orchestration
│   │   └── DTO / Mapper'lar
│   │
│   └── Teknik Alt Yapı
│       ├── Middleware
│       ├── Logging / Audit
│       └── Caching Interface
│
├── 3) İş / Domain Katmanı
│   ├── Domain Modelleri
│   │   ├── Entity / Value Object
│   │   └── Aggregate
│   │
│   ├── İş Servisleri
│   │   ├── Domain Services
│   │   ├── Factories
│   │   └── Policy / Rules
│   │
│   ├── Domain Event'ler
│   │   ├── Event Publishing
│   │   ├── Event Handlers
│   │   └── Integration Events
│   │
│   └── DDD Pattern'leri
│       ├── Bounded Context
```

```
│       ├── Repository Interface
│       └── CQRS / Command-Query
│
├── 4) Veri Erişim / Persistence Layer
│   ├── Repository & DAO
│   │   ├── CRUD Methods
│   │   └── Query Builders
│   │
│   ├── ORM / SDK
│   │   ├── Entity Framework
│   │   ├── Hibernate
│   │   └── Sequelize / Prisma
│   │
│   ├── Cache / Temp Store
│   │   ├── Redis
│   │   └── In-Memory
│   │
│   └── Data Abstraction
│       ├── Interfaces
│       └── Adapter Pattern
│
├── 5) Veri / Database / Storage
│   ├── SQL Veritabanı
│   │   ├── Tablo, İlişki
│   │   ├── Index, View
│   │   └── Stored Procedures
│   │
│   ├── NoSQL
│   │   ├── Document DB
│   │   ├── Key-Value
│   │   └── Graph DB
│   │
│   ├── Blob / File Storage
│   │   ├── Cloud Storage
│   │   └── File System
│   │
│   └── Data Lake / Warehouse
│
├── 6) Servis & Entegrasyon Katmanı
│   ├── Microservices
│   │   ├── Bounded Context bazlı servis
│   │   ├── Event Bus
│   │   └── Service Discovery
│   │
│   ├── API Gateway
│   ├── External API Integrations
│   │   ├── HTTP Clients
│   │   └── Authentication
```

```
│   │
│   └── Messaging
│       ├── Kafka / RabbitMQ
│       └── Pub/Sub
│
├── 7) Infra / DevOps
│   ├── Deployment
│   │   ├── Container / Orchestrator
│   │   └── CI/CD
│   │
│   ├── Monitoring
│   │   ├── Logs
│   │   └── Metrics
│   │
│   └── Security
│       ├── Auth/Zauth
│       └── Secret Management
│
└── 8) Pattern / Stil Katmanları
├── Layered (N-Tier) – Tipik üç katman: UI, Logic, Data 1
├── Onion / Hexagonal – Domain merkezli herkes adapter üzerinden konuşur 2
├── Microkernel / Plugin
├── Event-Driven
├── CQRS / Event Sourcing
└── Serverless / Cloud Native
```

Bu planlara bak ,sahada derinlere in ağaçlar dallansin eksikleri bul

```
UYGULAMA MİMARİSİ
├─ 0) ÜRÜN / KANAL
│  ├─ Web
│  ├─ Mobil
│  ├─ Desktop
│  ├─ CLI / Admin panel
│  └─ Public API / Partner API
│
├─ 1) İSTEMCİ / SUNUM KATMANI
│  ├─ Ekranlar / Sayfalar
│  ├─ Komponentler / Widget'lar
│  ├─ State management
│  ├─ Routing / Navigation
│  ├─ Formlar / Input validation
│  ├─ Tema / Stil / UI kit
│  ├─ Dil desteği / i18n
│  ├─ Erişilebilirlik / accessibility
│  ├─ Hata gösterimi / empty state / loading state
│  ├─ Oturum ekranları / login / logout / refresh
│  └─ Offline cache / local storage / analytics hooks
```

```
│
├─ 2) GİRİŞ KATMANI / EDGE
│   ├─ API Gateway
│   ├─ BFF (Backend for Frontend)
│   ├─ Controller / Handler / Resolver
│   ├─ Request parsing
│   ├─ Response shaping
│   ├─ AuthN / AuthZ
│   ├─ Rate limit / throttle
│   ├─ Versioning
│   ├─ Pagination / filtering / sorting
│   └─ CORS / CSRF / anti-abuse
│
├─ 3) UYGULAMA KATMANI
│   ├─ Use case'ler
│   ├─ İş akışı orkestrasyonu
│   ├─ DTO / Mapper
│   ├─ Transaction yönetimi
│   ├─ Idempotency
│   ├─ Retry / fallback
│   ├─ Saga / process manager
│   └─ Uygulama servisleri
│
├─ 4) DOMAIN / İŞ KATMANI
│   ├─ Entity
│   ├─ Value Object
│   ├─ Aggregate
│   ├─ Domain Service
│   ├─ Domain Event
│   ├─ Policy / Rule / Specification
│   ├─ Factory
│   ├─ Bounded Context
│   └─ Domain invariants
│
├─ 5) VERİ ERİŞİM KATMANI
│   ├─ Repository
│   ├─ DAO
│   ├─ ORM
│   ├─ Query builder
│   ├─ Mapper / adapter
│   ├─ Migration
│   ├─ Seed / bootstrap data
│   ├─ Cache access
│   ├─ Search index access
│   └─ File / blob access
│
├─ 6) ENTEGRASYON / ASENKRON KATMAN
│   ├─ Message queue
```

```
│   ├─ Event bus
│   ├─ Pub/Sub
│   ├─ Background jobs / workers
│   ├─ Scheduler / cron
│   ├─ Webhook
│   ├─ External API client
│   ├─ Outbox / inbox pattern
│   └─ Notification dispatch
│
├─ 7) ALTYAPI / INFRA
│   ├─ Veritabanı
│   ├─ Cache sunucusu
│   ├─ Dosya / object storage
│   ├─ Search engine
│   ├─ Message broker
│   ├─ Secrets / config
│   ├─ Container / orchestration
│   ├─ CDN / DNS / load balancer
│   └─ Network / firewall
│
├─ 8) KESİŞEN ENDİŞELER
│   ├─ Logging
│   ├─ Metrics
│   ├─ Tracing
│   ├─ Monitoring / alerting
│   ├─ Error handling
│   ├─ Security
│   ├─ Audit trail
│   ├─ Encryption
│   ├─ Feature flags
│   ├─ Observability
│   └─ Backup / restore
│
├─ 9) YAŞAM DÖNGÜSÜ / OPERASYON
│   ├─ Testler
│   │   ├─ Unit
│   │   ├─ Integration
│   │   ├─ E2E
│   │   └─ Contract tests
│   ├─ CI / CD
│   ├─ Release / rollout
│   ├─ Rollback
│   ├─ Incident management
│   └─ Maintenance
│
└─ 10) MİMARİ STİL / YAPI TÜRÜ
    ├─ Layered / N-Tier
    ├─ Clean Architecture
```

```
  ├─ Onion Architecture
  ├─ Hexagonal Architecture
  ├─ Modular Monolith
  ├─ Microservices
  ├─ Event-Driven
  ├─ Serverless
  └─ CQRS / Event Sourcing


E-Ticaret
├─ Ürün kataloğu
├─ Sepet
├─ Sipariş
├─ Ödeme
├─ Kargo
├─ Stok
├─ Kupon / kampanya
├─ Bildirim
└─ İade / refund

Sosyal Uygulama
├─ Feed
├─ Post / yorum / beğeni
├─ Takip sistemi
├─ Medya yükleme
├─ Moderasyon
├─ Bildirim
├─ Mesajlaşma
└─ Öneri sistemi

Fintech
├─ Kimlik doğrulama
├─ Hesap / cüzdan
├─ Ledger / kayıt defteri
├─ Ödeme
├─ Risk / fraud
├─ Reconciliation
├─ Audit
└─ KYC / AML

UYGULAMA MİMARİSİ
├─ 0) ÜRÜN / KANAL
│   ├─ Web
│   │   ├─ SPA (React, Vue, Angular)
│   │   ├─ MPA (Server-side rendering)
│   │   └─ PWA
│   ├─ Mobil
```

```
│   │   ├─ Native (iOS, Android)
│   │   ├─ Cross-platform (Flutter, React Native)
│   │   └─ Hybrid (Cordova, Capacitor)
│   ├─ Desktop
│   │   ├─ Native App (Windows, macOS, Linux)
│   │   └─ Electron / Tauri
│   ├─ CLI / Admin panel
│   │   ├─ Terminal UI
│   │   └─ Admin Dashboard
│   └─ Public API / Partner API
│       ├─ REST
│       ├─ GraphQL
│       └─ gRPC
│
├─ 1) İSTEMCİ / SUNUM KATMANI
│   ├─ Ekranlar / Sayfalar
│   ├─ Komponentler / Widget'lar
│   ├─ State management
│   │   ├─ Redux / MobX
│   │   ├─ Context / Provider
│   │   └─ Vuex / Pinia
│   ├─ Routing / Navigation
│   │   ├─ SPA Router
│   │   └─ Deep linking
│   ├─ Formlar / Input validation
│   ├─ Tema / Stil / UI kit
│   │   ├─ CSS / SCSS / Tailwind
│   │   └─ Design Tokens
│   ├─ Dil desteği / i18n
│   ├─ Erişilebilirlik / accessibility
│   ├─ Hata gösterimi / empty state / loading state
│   ├─ Oturum ekranları / login / logout / refresh
│   └─ Offline cache / local storage / analytics hooks
│
├─ 2) GİRİŞ KATMANI / EDGE
│   ├─ API Gateway
│   │   ├─ Routing
│   │   ├─ Rate limiting
│   │   ├─ Auth / JWT validation
│   │   └─ Logging / Tracing
│   ├─ BFF (Backend for Frontend)
│   │   ├─ Response shaping
│   │   └─ Aggregation of services
│   ├─ Controller / Handler / Resolver
│   ├─ Request parsing / validation
│   ├─ Response shaping / formatting
│   ├─ AuthN / AuthZ
│   ├─ Rate limit / throttle
```

```
│   ├─ Versioning
│   ├─ Pagination / filtering / sorting
│   └─ CORS / CSRF / anti-abuse
│
├─ 3) UYGULAMA KATMANI
│   ├─ Use case'ler
│   ├─ İş akışı orkestrasyonu
│   ├─ DTO / Mapper
│   ├─ Transaction yönetimi
│   ├─ Idempotency
│   ├─ Retry / fallback
│   ├─ Saga / process manager
│   └─ Uygulama servisleri
│       ├─ Payment Service
│       ├─ Notification Service
│       └─ Order Orchestration
│
├─ 4) DOMAIN / İŞ KATMANI
│   ├─ Entity
│   ├─ Value Object
│   ├─ Aggregate
│   ├─ Domain Service
│   ├─ Domain Event
│   ├─ Policy / Rule / Specification
│   ├─ Factory
│   ├─ Bounded Context
│   └─ Domain invariants
│
├─ 5) VERİ ERİŞİM KATMANI
│   ├─ Repository
│   ├─ DAO
│   ├─ ORM
│   │   ├─ Entity Framework / Hibernate
│   │   └─ Prisma / Sequelize
│   ├─ Query builder
│   ├─ Mapper / adapter
│   ├─ Migration
│   ├─ Seed / bootstrap data
│   ├─ Cache access
│   ├─ Search index access
│   └─ File / blob access
│
├─ 6) ENTEGRASYON / ASENKRON KATMAN
│   ├─ Message queue
│   │   ├─ Kafka / RabbitMQ / SQS
│   │   └─ Dead letter queue
│   ├─ Event bus
│   ├─ Pub/Sub
```

```
│   ├─ Background jobs / workers
│   ├─ Scheduler / cron
│   ├─ Webhook
│   ├─ External API client
│   ├─ Outbox / inbox pattern
│   └─ Notification dispatch
│
├─ 7) ALTYAPI / INFRA
│   ├─ Veritabanı
│   │   ├─ SQL (Postgres, MySQL)
│   │   ├─ NoSQL (Mongo, DynamoDB)
│   │   └─ Graph DB
│   ├─ Cache sunucusu
│   │   ├─ Redis
│   │   └─ Memcached
│   ├─ Dosya / object storage
│   │   ├─ S3 / Azure Blob / GCS
│   │   └─ File System
│   ├─ Search engine
│   │   └─ Elasticsearch / OpenSearch
│   ├─ Message broker
│   ├─ Secrets / config
│   ├─ Container / orchestration
│   │   ├─ Docker
│   │   └─ Kubernetes
│   ├─ CDN / DNS / load balancer
│   └─ Network / firewall
│
├─ 8) KESİŞEN ENDİŞELER
│   ├─ Logging
│   ├─ Metrics
│   ├─ Tracing
│   ├─ Monitoring / alerting
│   ├─ Error handling
│   ├─ Security
│   ├─ Audit trail
│   ├─ Encryption
│   ├─ Feature flags
│   ├─ Observability
│   └─ Backup / restore
│
├─ 9) YAŞAM DÖNGÜSÜ / OPERASYON
│   ├─ Testler
│   │   ├─ Unit
│   │   ├─ Integration
│   │   ├─ E2E
│   │   └─ Contract tests
│   ├─ CI / CD
```

```
│   ├─ Release / rollout
│   ├─ Rollback
│   ├─ Incident management
│   └─ Maintenance
│
└─ 10) MİMARİ STİL / YAPI TÜRÜ
    ├─ Layered / N-Tier
    │   ├─ Sunum
    │   ├─ Uygulama
    │   └─ Veri
    ├─ Clean Architecture
    │   ├─ Entities
    │   ├─ Use Cases
    │   ├─ Interface Adapters
    │   └─ Frameworks & Drivers
    ├─ Onion Architecture
    │   ├─ Core
    │  ├─ Application
    │  ├─ Infrastructure
    │  └─ External
    ├─ Hexagonal Architecture
    │   ├─ Domain
    │   ├─ Ports
    │   ├─ Adapters
    │   └─ External Systems
    ├─ Modular Monolith
    │   ├─ Module A
    │   ├─ Module B
    │   └─ Shared Kernel
    ├─ Microservices
    │   ├─ Service A
    │   ├─ Service B
    │   ├─ Service C
    │   ├─ API Gateway
    │   └─ Event / Message Bus
    ├─ Event-Driven
    │   ├─ Event Producer
    │   ├─ Event Consumer
    │   └─ Event Store
    ├─ Serverless
    │   ├─ Functions
    │   ├─ Event Triggers
    │   └─ Managed Services
    └─ CQRS / Event Sourcing
        ├─ Command Model
        ├─ Query Model
        ├─ Event Store
        └─ Projection / Read Model
```

```
import React from "react";

const tree = {
  label: "UYGULAMA MİMARİSİ",
  children: [
    {
      label: "0) UYGULAMA TİPİ / ÜRÜN ALANI",
      children: [
        {
          label: "Web uygulaması",
          children: [
            { label: "Kurumsal web" },
            { label: "SaaS paneli" },
            { label: "E-ticaret" },
            { label: "Blog / içerik" },
            { label: "Portal / intranet" },
          ],
        },
        {
          label: "Mobil uygulama",
          children: [
            { label: "iOS / Android native" },
            { label: "Flutter / React Native" },
            { label: "Offline-first mobil" },
            { label: "Super-app alt modülü" },
          ],
        },
        {
          label: "Desktop uygulama",
          children: [
            { label: "Windows / macOS / Linux native" },
            { label: "Electron / Tauri" },
            { label: "Kurumsal masaüstü araç" },
          ],
        },
        {
          label: "Komut satırı / otomasyon",
          children: [
            { label: "CLI tool" },
            { label: "DevOps script aracı" },
            { label: "Batch processing" },
          ],
        },
        {
          label: "API / servis ürünü",
          children: [
            { label: "Public API" },
```

```
        { label: "Partner API" },
        { label: "Internal API" },
        { label: "Event-driven servis" },
      ],
    },
    {
      label: "Gerçek zamanlı sistem",
      children: [
        { label: "Chat / mesajlaşma" },
        { label: "Canlı skor / yayın" },
        { label: "Trading / market data" },
        { label: "Game backend" },
      ],
    },
    {
      label: "Veri yoğun sistem",
      children: [
        { label: "BI / dashboard" },
        { label: "Search platform" },
        { label: "AI destekli ürün" },
        { label: "Raporlama / analitik" },
      ],
    },
  ],
},

{
  label: "1) İSTEMCİ / SUNUM KATMANI",
  children: [
    {
      label: "Ekranlar / Sayfalar",
      children: [
        { label: "Home / landing" },
        { label: "Liste ekranı" },
        { label: "Detay ekranı" },
        { label: "Form ekranı" },
        { label: "Profil / ayarlar" },
        { label: "Admin panel" },
      ],
    },
    {
      label: "Komponentler / Widget'lar",
      children: [
        { label: "Button / input / select" },
        { label: "Modal / drawer" },
        { label: "Card / table / list" },
        { label: "Tabs / accordion" },
        { label: "Toast / snackbar" },
```

```
    { label: "Charts / graphs" },
  ],
},
{
  label: "State management",
  children: [
    { label: "Local state" },
    { label: "Global state" },
    { label: "Server state" },
    { label: "Derived state" },
    { label: "Cache state" },
    { label: "Session state" },
  ],
},
{
  label: "Routing / Navigation",
  children: [
    { label: "Route guards" },
    { label: "Nested routes" },
    { label: "Deep link" },
    { label: "Redirect" },
    { label: "Breadcrumb" },
  ],
},
{
  label: "Formlar / Input validation",
  children: [
    { label: "Client validation" },
    { label: "Server validation" },
    { label: "Mask / format" },
    { label: "Error messages" },
    { label: "Submit state" },
  ],
},
{
  label: "Tema / Stil / UI kit",
  children: [
    { label: "Design system" },
    { label: "Component library" },
    { label: "Theme switcher" },
    { label: "Responsive layout" },
    { label: "Design tokens" },
  ],
},
{
  label: "Dil desteği / i18n",
  children: [
    { label: "Locale detection" },
```

```
        { label: "Translation files" },
        { label: "Pluralization" },
        { label: "Date / number format" },
      ],
    },
    {
      label: "Erişilebilirlik / accessibility",
      children: [
        { label: "Keyboard navigation" },
        { label: "Screen reader labels" },
        { label: "Contrast" },
        { label: "Focus management" },
      ],
    },
    {
      label: "Hata / empty / loading state",
      children: [
        { label: "Skeleton" },
        { label: "Spinner" },
        { label: "Retry screen" },
        { label: "No data screen" },
        { label: "Offline screen" },
      ],
    },
    {
      label: "Oturum ekranları",
      children: [
        { label: "Login" },
        { label: "Logout" },
        { label: "Refresh token" },
        { label: "Forgot password" },
        { label: "2FA / MFA" },
      ],
    },
    {
      label: "Offline cache / local storage / analytics",
      children: [
        { label: "Local storage" },
        { label: "Session storage" },
        { label: "IndexedDB" },
        { label: "Crash analytics" },
        { label: "Usage analytics" },
      ],
    },
  ],
},

{
```

```
label: "2) GİRİŞ KATMANI / EDGE",
children: [
  {
    label: "API Gateway",
    children: [
      { label: "Routing" },
      { label: "Auth validation" },
      { label: "Rate limiting" },
      { label: "Request/response transform" },
      { label: "Logging" },
    ],
  },
  {
    label: "BFF (Backend for Frontend)",
    children: [
      { label: "Web BFF" },
      { label: "Mobile BFF" },
      { label: "Admin BFF" },
      { label: "Aggregation endpoint" },
    ],
  },
  {
    label: "Controller / Handler / Resolver",
    children: [
      { label: "REST controller" },
      { label: "GraphQL resolver" },
      { label: "gRPC handler" },
      { label: "CLI command handler" },
    ],
  },
  {
    label: "Request parsing / validation",
    children: [
      { label: "Schema validation" },
      { label: "DTO binding" },
      { label: "Sanitization" },
    ],
  },
  {
    label: "Response shaping / formatting",
    children: [
      { label: "DTO response" },
      { label: "Pagination payload" },
      { label: "Error payload" },
    ],
  },
  {
    label: "AuthN / AuthZ",
```

```
      children: [
        { label: "Session auth" },
        { label: "JWT" },
        { label: "OAuth2 / OpenID Connect" },
        { label: "RBAC / ABAC" },
      ],
    },
    { label: "Rate limit / throttle" },
    { label: "Versioning" },
    { label: "Pagination / filtering / sorting" },
    { label: "CORS / CSRF / anti-abuse" },
  ],
},

{
  label: "3) UYGULAMA KATMANI",
  children: [
    {
      label: "Use case'ler",
      children: [
        { label: "Create" },
        { label: "Update" },
        { label: "Delete" },
        { label: "Approve" },
        { label: "Cancel" },
        { label: "Publish" },
      ],
    },
    {
      label: "İş akışı orkestrasyonu",
      children: [
        { label: "Step orchestration" },
        { label: "Conditional flow" },
        { label: "Compensation flow" },
      ],
    },
    {
      label: "DTO / Mapper",
      children: [
        { label: "Request DTO" },
        { label: "Response DTO" },
        { label: "Command / Query model" },
        { label: "Mapping rules" },
      ],
    },
    {
      label: "Transaction yönetimi",
      children: [
```

```
        { label: "Atomic transaction" },
        { label: "Isolation level" },
        { label: "Commit / rollback" },
      ],
    },
    {
      label: "Idempotency",
      children: [
        { label: "Idempotency key" },
        { label: "Duplicate prevention" },
        { label: "Replay protection" },
      ],
    },
    {
      label: "Retry / fallback",
      children: [
        { label: "Retry policy" },
        { label: "Circuit breaker" },
        { label: "Fallback response" },
      ],
    },
    {
      label: "Saga / process manager",
      children: [
        { label: "Orchestrated saga" },
        { label: "Choreographed saga" },
        { label: "Compensation action" },
      ],
    },
    {
      label: "Uygulama servisleri",
      children: [
        { label: "Order service" },
        { label: "Payment service" },
        { label: "Notification service" },
        { label: "Search service" },
      ],
    },
  ],
},

{
  label: "4) DOMAIN / İŞ KATMANI",
  children: [
    {
      label: "Entity",
      children: [
        { label: "Identity" },
```

```
    { label: "Lifecycle" },
    { label: "State transitions" },
  ],
},
{
  label: "Value Object",
  children: [
    { label: "Immutable data" },
    { label: "Validation" },
    { label: "Equality by value" },
  ],
},
{
  label: "Aggregate",
  children: [
    { label: "Aggregate root" },
    { label: "Consistency boundary" },
    { label: "Child entities" },
  ],
},
{
  label: "Domain Service",
  children: [
    { label: "Business calculation" },
    { label: "Cross-entity rule" },
    { label: "Policy enforcement" },
  ],
},
{
  label: "Domain Event",
  children: [
    { label: "Event generation" },
    { label: "Event reaction" },
    { label: "Integration event" },
  ],
},
{
  label: "Policy / Rule / Specification",
  children: [
    { label: "Rule engine" },
    { label: "Invariant" },
    { label: "Eligibility check" },
  ],
},
{
  label: "Factory",
  children: [
    { label: "Object creation" },
```

```
        { label: "Complex construction" },
        { label: "Domain-safe creation" },
      ],
    },
    { label: "Bounded Context" },
    { label: "Domain invariants" },
  ],
},

{
  label: "5) VERİ ERİŞİM KATMANI",
  children: [
    {
      label: "Repository",
      children: [
        { label: "Read repository" },
        { label: "Write repository" },
        { label: "Custom queries" },
      ],
    },
    { label: "DAO" },
    {
      label: "ORM",
      children: [
        { label: "Entity mapping" },
        { label: "Lazy / eager load" },
        { label: "Change tracking" },
      ],
    },
    {
      label: "Query builder",
      children: [
        { label: "Dynamic filter" },
        { label: "Join" },
        { label: "Sorting" },
      ],
    },
    { label: "Mapper / adapter" },
    { label: "Migration" },
    { label: "Seed / bootstrap data" },
    {
      label: "Cache access",
      children: [
        { label: "Read-through" },
        { label: "Write-through" },
        { label: "Cache invalidation" },
      ],
    },
```

```
    { label: "Search index access" },
    { label: "File / blob access" },
  ],
},

{
  label: "6) ENTEGRASYON / ASENKRON KATMAN",
  children: [
    {
      label: "Message queue",
      children: [
        { label: "Kafka" },
        { label: "RabbitMQ" },
        { label: "SQS" },
        { label: "Dead letter queue" },
      ],
    },
    { label: "Event bus" },
    { label: "Pub/Sub" },
    {
      label: "Background jobs / workers",
      children: [
        { label: "Queue worker" },
        { label: "Batch worker" },
        { label: "Scheduled worker" },
      ],
    },
    {
      label: "Scheduler / cron",
      children: [
        { label: "Periodic sync" },
        { label: "Cleanup job" },
        { label: "Report job" },
      ],
    },
    { label: "Webhook" },
    { label: "External API client" },
    { label: "Outbox / inbox pattern" },
    { label: "Notification dispatch" },
  ],
},

{
  label: "7) ALTYAPI / INFRA",
  children: [
    {
      label: "Veritabanı",
      children: [
```

```
        { label: "SQL → Postgres / MySQL / MSSQL" },
        { label: "NoSQL → Mongo / DynamoDB" },
        { label: "Graph DB" },
        { label: "Time-series DB" },
      ],
    },
    {
      label: "Cache sunucusu",
      children: [
        { label: "Redis" },
        { label: "Memcached" },
      ],
    },
    {
      label: "Dosya / object storage",
      children: [
        { label: "S3 / Azure Blob / GCS" },
        { label: "File system" },
      ],
    },
    {
      label: "Search engine",
      children: [
        { label: "Elasticsearch" },
        { label: "OpenSearch" },
      ],
    },
    { label: "Message broker" },
    { label: "Secrets / config" },
    {
      label: "Container / orchestration",
      children: [
        { label: "Docker" },
        { label: "Kubernetes" },
        { label: "Helm / manifests" },
      ],
    },
    { label: "CDN / DNS / load balancer" },
    { label: "Network / firewall" },
  ],
},

{
  label: "8) KESİŞEN ENDİŞELER",
  children: [
    {
      label: "Logging / Metrics / Tracing",
      children: [
```

```
        { label: "Structured logs" },
        { label: "Correlation id" },
        { label: "Metrics" },
        { label: "Distributed tracing" },
      ],
    },
    {
      label: "Monitoring / alerting",
      children: [
        { label: "Dashboard" },
        { label: "Alert rule" },
        { label: "On-call" },
      ],
    },
    {
      label: "Error handling",
      children: [
        { label: "Exception mapping" },
        { label: "Retryable error" },
        { label: "User-friendly message" },
      ],
    },
    {
      label: "Security",
      children: [
        { label: "Auth" },
        { label: "Authorization" },
        { label: "Secrets" },
        { label: "Threat mitigation" },
      ],
    },
    { label: "Audit trail" },
    { label: "Encryption" },
    { label: "Feature flags" },
    { label: "Observability" },
    { label: "Backup / restore" },
  ],
},

{
  label: "9) YAŞAM DÖNGÜSÜ / OPERASYON",
  children: [
    {
      label: "Testler",
      children: [
        { label: "Unit" },
        { label: "Integration" },
        { label: "E2E" },
```

```
        { label: "Contract tests" },
        { label: "Smoke test" },
      ],
    },
    { label: "CI / CD" },
    { label: "Release / rollout" },
    { label: "Rollback" },
    { label: "Incident management" },
    { label: "Maintenance" },
  ],
},

{
  label: "10) MİMARİ STİL / YAPI TÜRÜ",
  children: [
    {
      label: "Layered / N-Tier",
      children: [
        { label: "Sunum" },
        { label: "Uygulama" },
        { label: "Domain" },
        { label: "Veri erişim" },
        { label: "Veri" },
      ],
    },
    {
      label: "Clean Architecture",
      children: [
        { label: "Entities" },
        { label: "Use Cases" },
        { label: "Interface Adapters" },
        { label: "Frameworks & Drivers" },
      ],
    },
    {
      label: "Onion Architecture",
      children: [
        { label: "Core" },
        { label: "Application" },
        { label: "Infrastructure" },
        { label: "External systems" },
      ],
    },
    {
      label: "Hexagonal Architecture",
      children: [
        { label: "Domain" },
        { label: "Ports" },
```

```
    { label: "Adapters" },
    { label: "Outside world" },
  ],
},
{
  label: "Modular Monolith",
  children: [
    { label: "Module A" },
    { label: "Module B" },
    { label: "Module C" },
    { label: "Shared kernel" },
    { label: "Internal events" },
  ],
},
{
  label: "Microservices",
  children: [
    { label: "Service A" },
    { label: "Service B" },
    { label: "Service C" },
    { label: "API Gateway" },
    { label: "Event / Message Bus" },
    { label: "Service discovery" },
  ],
},
{
  label: "Event-Driven",
  children: [
    { label: "Producer" },
    { label: "Consumer" },
    { label: "Event broker" },
    { label: "Event store" },
    { label: "Projection" },
  ],
},
{
  label: "Serverless",
  children: [
    { label: "Functions" },
    { label: "Event triggers" },
    { label: "Managed services" },
    { label: "Cold start" },
  ],
},
{
  label: "CQRS / Event Sourcing",
  children: [
    { label: "Command model" },
```

```
            { label: "Query model" },
            { label: "Event store" },
            { label: "Projection / read model" },
            { label: "Snapshot" },
          ],
        },
      ],
    },
  ],
};

function Node({ node, depth = 0 }) {
  const isRoot = depth === 0;
  return (
    <div className={depth === 0 ? "w-full" : "pl-5 border-l border-slate-200
dark:border-slate-700"}>
      <div
        className={`inline-flex items-center rounded-2xl px-4 py-2 shadow-sm border ${
          isRoot
            ? "bg-slate-900 text-white border-slate-900 text-lg font-semibold"
            : "bg-white dark:bg-slate-900 text-slate-900 dark:text-slate-100 border-slate-200
dark:border-slate-700"
        }`}
      >
        <span>{node.label}</span>
      </div>
      {node.children?.length ? (
        <div className="mt-4 space-y-4">
          {node.children.map((child, i) => (
            <Node key={i} node={child} depth={depth + 1} />
          ))}
        </div>
      ) : null}
    </div>
  );
}

export default function App() {
  return (
    <div className="min-h-screen bg-gradient-to-b from-slate-50 to-slate-100
dark:from-slate-950 dark:to-slate-900 p-4 md:p-8">
      <div className="mx-auto max-w-6xl">
        <div className="mb-6">
          <h1 className="text-3xl md:text-5xl font-bold tracking-tight text-slate-900
dark:text-white">
            Uygulama Mimarisi Ağacı
          </h1>
          <p className="mt-2 text-slate-600 dark:text-slate-300">
```

```
          Her uygulama tipine uyarlanabilecek, katmanlı ve dallı mimari şema.
        </p>
      </div>

      <div className="rounded-3xl border border-slate-200 dark:border-slate-800
bg-white/80 dark:bg-slate-9


import React from "react";

const tree = { label: "UYGULAMA MİMARİSİ", children: [ { label: "0) UYGULAMA TİPİ /
ÜRÜN ALANI", children: [ { label: "Web uygulaması", children: [ { label: "Kurumsal web" }, {
label: "SaaS paneli" }, { label: "E-ticaret" }, { label: "Blog / içerik" }, { label: "Portal / intranet" },
], }, { label: "Mobil uygulama", children: [ { label: "iOS / Android native" }, { label: "Flutter /
React Native" }, { label: "Offline-first mobil" }, { label: "Super-app alt modülü" }, ], }, { label:
"Desktop uygulama", children: [ { label: "Windows / macOS / Linux native" }, { label: "Electron
/ Tauri" }, { label: "Kurumsal masaüstü araç" }, ], }, { label: "Komut satırı / otomasyon",
children: [ { label: "CLI tool" }, { label: "DevOps script aracı" }, { label: "Batch processing" }, ],
}, { label: "API / servis ürünü", children: [ { label: "Public API" }, { label: "Partner API" }, { label:
"Internal API" }, { label: "Event-driven servis" }, ], }, { label: "Gerçek zamanlı sistem", children:
[ { label: "Chat / mesajlaşma" }, { label: "Canlı skor / yayın" }, { label: "Trading / market data"
}, { label: "Game backend" }, ], }, { label: "Veri yoğun sistem", children: [ { label: "BI /
dashboard" }, { label: "Search platform" }, { label: "AI destekli ürün" }, { label: "Raporlama /
analitik" }, ], }, ], },

{
  label: "1) İSTEMCİ / SUNUM KATMANI",
  children: [
    {
      label: "Ekranlar / Sayfalar",
      children: [
        { label: "Home / landing" },
        { label: "Liste ekranı" },
        { label: "Detay ekranı" },
        { label: "Form ekranı" },
        { label: "Profil / ayarlar" },
        { label: "Admin panel" },
      ],
    },
    {
      label: "Komponentler / Widget'lar",
      children: [
        { label: "Button / input / select" },
        { label: "Modal / drawer" },
        { label: "Card / table / list" },
        { label: "Tabs / accordion" },
        { label: "Toast / snackbar" },
        { label: "Charts / graphs" },
```

```
  ],
},
{
  label: "State management",
  children: [
    { label: "Local state" },
    { label: "Global state" },
    { label: "Server state" },
    { label: "Derived state" },
    { label: "Cache state" },
    { label: "Session state" },
  ],
},
{
  label: "Routing / Navigation",
  children: [
    { label: "Route guards" },
    { label: "Nested routes" },
    { label: "Deep link" },
    { label: "Redirect" },
    { label: "Breadcrumb" },
  ],
},
{
  label: "Formlar / Input validation",
  children: [
    { label: "Client validation" },
    { label: "Server validation" },
    { label: "Mask / format" },
    { label: "Error messages" },
    { label: "Submit state" },
  ],
},
{
  label: "Tema / Stil / UI kit",
  children: [
    { label: "Design system" },
    { label: "Component library" },
    { label: "Theme switcher" },
    { label: "Responsive layout" },
    { label: "Design tokens" },
  ],
},
{
  label: "Dil desteği / i18n",
  children: [
    { label: "Locale detection" },
    { label: "Translation files" },
```

```
        { label: "Pluralization" },
        { label: "Date / number format" },
      ],
    },
    {
      label: "Erişilebilirlik / accessibility",
      children: [
        { label: "Keyboard navigation" },
        { label: "Screen reader labels" },
        { label: "Contrast" },
        { label: "Focus management" },
      ],
    },
    {
      label: "Hata / empty / loading state",
      children: [
        { label: "Skeleton" },
        { label: "Spinner" },
        { label: "Retry screen" },
        { label: "No data screen" },
        { label: "Offline screen" },
      ],
    },
    {
      label: "Oturum ekranları",
      children: [
        { label: "Login" },
        { label: "Logout" },
        { label: "Refresh token" },
        { label: "Forgot password" },
        { label: "2FA / MFA" },
      ],
    },
    {
      label: "Offline cache / local storage / analytics",
      children: [
        { label: "Local storage" },
        { label: "Session storage" },
        { label: "IndexedDB" },
        { label: "Crash analytics" },
        { label: "Usage analytics" },
      ],
    },
  ],
},

{
  label: "2) GİRİŞ KATMANI / EDGE",
```

```
children: [
  {
    label: "API Gateway",
    children: [
      { label: "Routing" },
      { label: "Auth validation" },
      { label: "Rate limiting" },
      { label: "Request/response transform" },
      { label: "Logging" },
    ],
  },
  {
    label: "BFF (Backend for Frontend)",
    children: [
      { label: "Web BFF" },
      { label: "Mobile BFF" },
      { label: "Admin BFF" },
      { label: "Aggregation endpoint" },
    ],
  },
  {
    label: "Controller / Handler / Resolver",
    children: [
      { label: "REST controller" },
      { label: "GraphQL resolver" },
      { label: "gRPC handler" },
      { label: "CLI command handler" },
    ],
  },
  {
    label: "Request parsing / validation",
    children: [
      { label: "Schema validation" },
      { label: "DTO binding" },
      { label: "Sanitization" },
    ],
  },
  {
    label: "Response shaping / formatting",
    children: [
      { label: "DTO response" },
      { label: "Pagination payload" },
      { label: "Error payload" },
    ],
  },
  {
    label: "AuthN / AuthZ",
    children: [
```

```
        { label: "Session auth" },
        { label: "JWT" },
        { label: "OAuth2 / OpenID Connect" },
        { label: "RBAC / ABAC" },
      ],
    },
    { label: "Rate limit / throttle" },
    { label: "Versioning" },
    { label: "Pagination / filtering / sorting" },
    { label: "CORS / CSRF / anti-abuse" },
  ],
},

{
  label: "3) UYGULAMA KATMANI",
  children: [
    {
      label: "Use case'ler",
      children: [
        { label: "Create" },
        { label: "Update" },
        { label: "Delete" },
        { label: "Approve" },
        { label: "Cancel" },
        { label: "Publish" },
      ],
    },
    {
      label: "İş akışı orkestrasyonu",
      children: [
        { label: "Step orchestration" },
        { label: "Conditional flow" },
        { label: "Compensation flow" },
      ],
    },
    {
      label: "DTO / Mapper",
      children: [
        { label: "Request DTO" },
        { label: "Response DTO" },
        { label: "Command / Query model" },
        { label: "Mapping rules" },
      ],
    },
    {
      label: "Transaction yönetimi",
      children: [
        { label: "Atomic transaction" },
```

```
        { label: "Isolation level" },
        { label: "Commit / rollback" },
      ],
    },
    {
      label: "Idempotency",
      children: [
        { label: "Idempotency key" },
        { label: "Duplicate prevention" },
        { label: "Replay protection" },
      ],
    },
    {
      label: "Retry / fallback",
      children: [
        { label: "Retry policy" },
        { label: "Circuit breaker" },
        { label: "Fallback response" },
      ],
    },
    {
      label: "Saga / process manager",
      children: [
        { label: "Orchestrated saga" },
        { label: "Choreographed saga" },
        { label: "Compensation action" },
      ],
    },
    {
      label: "Uygulama servisleri",
      children: [
        { label: "Order service" },
        { label: "Payment service" },
        { label: "Notification service" },
        { label: "Search service" },
      ],
    },
  ],
},

{
  label: "4) DOMAIN / İŞ KATMANI",
  children: [
    {
      label: "Entity",
      children: [
        { label: "Identity" },
        { label: "Lifecycle" },
```

```
    { label: "State transitions" },
  ],
},
{
  label: "Value Object",
  children: [
    { label: "Immutable data" },
    { label: "Validation" },
    { label: "Equality by value" },
  ],
},
{
  label: "Aggregate",
  children: [
    { label: "Aggregate root" },
    { label: "Consistency boundary" },
    { label: "Child entities" },
  ],
},
{
  label: "Domain Service",
  children: [
    { label: "Business calculation" },
    { label: "Cross-entity rule" },
    { label: "Policy enforcement" },
  ],
},
{
  label: "Domain Event",
  children: [
    { label: "Event generation" },
    { label: "Event reaction" },
    { label: "Integration event" },
  ],
},
{
  label: "Policy / Rule / Specification",
  children: [
    { label: "Rule engine" },
    { label: "Invariant" },
    { label: "Eligibility check" },
  ],
},
{
  label: "Factory",
  children: [
    { label: "Object creation" },
    { label: "Complex construction" },
```

```
        { label: "Domain-safe creation" },
      ],
    },
    { label: "Bounded Context" },
    { label: "Domain invariants" },
  ],
},

{
  label: "5) VERİ ERİŞİM KATMANI",
  children: [
    {
      label: "Repository",
      children: [
        { label: "Read repository" },
        { label: "Write repository" },
        { label: "Custom queries" },
      ],
    },
    { label: "DAO" },
    {
      label: "ORM",
      children: [
        { label: "Entity mapping" },
        { label: "Lazy / eager load" },
        { label: "Change tracking" },
      ],
    },
    {
      label: "Query builder",
      children: [
        { label: "Dynamic filter" },
        { label: "Join" },
        { label: "Sorting" },
      ],
    },
    { label: "Mapper / adapter" },
    { label: "Migration" },
    { label: "Seed / bootstrap data" },
    {
      label: "Cache access",
      children: [
        { label: "Read-through" },
        { label: "Write-through" },
        { label: "Cache invalidation" },
      ],
    },
    { label: "Search index access" },
```

```
    { label: "File / blob access" },
  ],
},

{
  label: "6) ENTEGRASYON / ASENKRON KATMAN",
  children: [
    {
      label: "Message queue",
      children: [
        { label: "Kafka" },
        { label: "RabbitMQ" },
        { label: "SQS" },
        { label: "Dead letter queue" },
      ],
    },
    { label: "Event bus" },
    { label: "Pub/Sub" },
    {
      label: "Background jobs / workers",
      children: [
        { label: "Queue worker" },
        { label: "Batch worker" },
        { label: "Scheduled worker" },
      ],
    },
    {
      label: "Scheduler / cron",
      children: [
        { label: "Periodic sync" },
        { label: "Cleanup job" },
        { label: "Report job" },
      ],
    },
    { label: "Webhook" },
    { label: "External API client" },
    { label: "Outbox / inbox pattern" },
    { label: "Notification dispatch" },
  ],
},

{
  label: "7) ALTYAPI / INFRA",
  children: [
    {
      label: "Veritabanı",
      children: [
        { label: "SQL → Postgres / MySQL / MSSQL" },
```

```
        { label: "NoSQL → Mongo / DynamoDB" },
        { label: "Graph DB" },
        { label: "Time-series DB" },
      ],
    },
    {
      label: "Cache sunucusu",
      children: [
        { label: "Redis" },
        { label: "Memcached" },
      ],
    },
    {
      label: "Dosya / object storage",
      children: [
        { label: "S3 / Azure Blob / GCS" },
        { label: "File system" },
      ],
    },
    {
      label: "Search engine",
      children: [
        { label: "Elasticsearch" },
        { label: "OpenSearch" },
      ],
    },
    { label: "Message broker" },
    { label: "Secrets / config" },
    {
      label: "Container / orchestration",
      children: [
        { label: "Docker" },
        { label: "Kubernetes" },
        { label: "Helm / manifests" },
      ],
    },
    { label: "CDN / DNS / load balancer" },
    { label: "Network / firewall" },
  ],
},

{
  label: "8) KESİŞEN ENDİŞELER",
  children: [
    {
      label: "Logging / Metrics / Tracing",
      children: [
        { label: "Structured logs" },
```

```
        { label: "Correlation id" },
        { label: "Metrics" },
        { label: "Distributed tracing" },
      ],
    },
    {
      label: "Monitoring / alerting",
      children: [
        { label: "Dashboard" },
        { label: "Alert rule" },
        { label: "On-call" },
      ],
    },
    {
      label: "Error handling",
      children: [
        { label: "Exception mapping" },
        { label: "Retryable error" },
        { label: "User-friendly message" },
      ],
    },
    {
      label: "Security",
      children: [
        { label: "Auth" },
        { label: "Authorization" },
        { label: "Secrets" },
        { label: "Threat mitigation" },
      ],
    },
    { label: "Audit trail" },
    { label: "Encryption" },
    { label: "Feature flags" },
    { label: "Observability" },
    { label: "Backup / restore" },
  ],
},

{
  label: "9) YAŞAM DÖNGÜSÜ / OPERASYON",
  children: [
    {
      label: "Testler",
      children: [
        { label: "Unit" },
        { label: "Integration" },
        { label: "E2E" },
        { label: "Contract tests" },
```

```
        { label: "Smoke test" },
      ],
    },
    { label: "CI / CD" },
    { label: "Release / rollout" },
    { label: "Rollback" },
    { label: "Incident management" },
    { label: "Maintenance" },
  ],
},

{
  label: "10) MİMARİ STİL / YAPI TÜRÜ",
  children: [
    {
      label: "Layered / N-Tier",
      children: [
        { label: "Sunum" },
        { label: "Uygulama" },
        { label: "Domain" },
        { label: "Veri erişim" },
        { label: "Veri" },
      ],
    },
    {
      label: "Clean Architecture",
      children: [
        { label: "Entities" },
        { label: "Use Cases" },
        { label: "Interface Adapters" },
        { label: "Frameworks & Drivers" },
      ],
    },
    {
      label: "Onion Architecture",
      children: [
        { label: "Core" },
        { label: "Application" },
        { label: "Infrastructure" },
        { label: "External systems" },
      ],
    },
    {
      label: "Hexagonal Architecture",
      children: [
        { label: "Domain" },
        { label: "Ports" },
        { label: "Adapters" },
```

```
    { label: "Outside world" },
  ],
},
{
  label: "Modular Monolith",
  children: [
    { label: "Module A" },
    { label: "Module B" },
    { label: "Module C" },
    { label: "Shared kernel" },
    { label: "Internal events" },
  ],
},
{
  label: "Microservices",
  children: [
    { label: "Service A" },
    { label: "Service B" },
    { label: "Service C" },
    { label: "API Gateway" },
    { label: "Event / Message Bus" },
    { label: "Service discovery" },
  ],
},
{
  label: "Event-Driven",
  children: [
    { label: "Producer" },
    { label: "Consumer" },
    { label: "Event broker" },
    { label: "Event store" },
    { label: "Projection" },
  ],
},
{
  label: "Serverless",
  children: [
    { label: "Functions" },
    { label: "Event triggers" },
    { label: "Managed services" },
    { label: "Cold start" },
  ],
},
{
  label: "CQRS / Event Sourcing",
  children: [
    { label: "Command model" },
    { label: "Query model" },
```

```
        { label: "Event store" },
        { label: "Projection / read model" },
        { label: "Snapshot" },
      ],
    },
  ],
},

], };

function Node({ node, depth = 0 }) { const isRoot = depth === 0; return ( <div
className={depth === 0 ? "w-full" : "pl-5 border-l border-slate-200 dark:border-slate-700"}>
<div className={inline-flex items-center rounded-2xl px-4 py-2 shadow-sm border ${ isRoot
? "bg-slate-900 text-white border-slate-900 text-lg font-semibold" : "bg-white
dark:bg-slate-900 text-slate-900 dark:text-slate-100 border-slate-200 dark:border-slate-700"
}} > <span>{node.label}</span> </div> {node.children?.length ? ( <div className="mt-4
space-y-4"> {node.children.map((child, i) => ( <Node key={i} node={child} depth={depth + 1}
/> ))} </div> ) : null} </div> ); }

export default function App() { return ( <div className="min-h-screen bg-gradient-to-b
from-slate-50 to-slate-100 dark:from-slate-950 dark:to-slate-900 p-4 md:p-8"> <div
className="mx-auto max-w-6xl"> <div className="mb-6"> <h1 className="text-3xl
md:text-5xl font-bold tracking-tight text-slate-900 dark:text-white"> Uygulama Mimarisi Ağacı
</h1> <p className="mt-2 text-slate-600 dark:text-slate-300"> Her uygulama tipine
uyarlanabilecek, katmanlı ve dallı mimari şema. </p> </div>

<div className="rounded-3xl border border-slate-200 dark:border-slate-800 bg-white/80
dark:bg-slate-950/60 p-5 md:p-8 shadow-xl overflow-auto">
      <Node node={tree} />
    </div>
  </div>
</div>

); }
```

Aynen, önce işin mantığını kafanda netleştirelim. 🌪️
Yani her dalın ayrı ayrı nasıl planlanacağını planlayacağız, sonra o plan üzerinden asıl dal planlamasına geçeceğiz.

Bunu 3 aşamada yapabiliriz:

Aşama 0: Planlamayı planlamak (Meta-plan)

1. Hedef Belirleme

Hangi uygulama tipi (Web, Mobil, Microservice, Serverless…)

Hangi kullanıcı tipleri ve kanallar

Hangi entegrasyonlar ve altyapı zorunlu

2. Katman Bazlı Planlama

Her ana katmanı tek tek ele al

Her katmanda olması gereken alt parçaları listelerken:

Zorunlu / opsiyonel

Bağımlılıklar (hangi katman önce gelmeli)

Veri akışı ve kontrol akışı

3. Derinlik ve Atomizasyon

Alt alt katmanlara kadar git (entity > value object > domain service gibi)

Her bir atomik parça: sorumluluk + ilişkisi + tercih senaryosu

4. Sıra ve Mantık

Hangi katman önce kurulacak / hangisi sonra

Deploy ve test önceliği

Asenkron ve event-driven öğelerin konumu

5. Operasyonel Planlama

Logging, monitoring, CI/CD, rollback

Hangi katmanlarda hangi kesişen endişe uygulanacak

Backup ve güvenlik prosedürleri

Aşama 1: Her dalı ayrı ayrı planlamak

0) Ürün / Kanal → Hangi platformlar var, her biri için UI/UX ayrı mı, API gerekli mi?

1) Sunum / İstemci Katmanı → Hangi framework, state management tipi, routing, form validation

2) Giriş / Edge → API Gateway, auth, throttle, request/response shaping

3) Uygulama Katmanı → Use case, transaction, retry/fallback, orchestration

4) Domain Katmanı → Entity, value object, aggregate, domain service, event

5) Veri Erişim Katmanı → Repository, ORM, mapper, query builder

6) Entegrasyon / Asenkron → MQ, Event bus, scheduler, webhook, notification

7) Infra / Altyapı → DB, cache, file storage, broker, network

8) Kesişen Endişeler → Logging, monitoring, security, audit, backup

9) Yaşam Döngüsü / Operasyon → Test, CI/CD, rollout, rollback, incident

10) Mimari Stil / Yapı → Katmanlı, clean, microservice, event-driven, serverless

---

Aşama 2: Ana planlama (detaylı ağaç)

Her dal için:

Alt dallar + alt alt dallar

Sorumluluk + veri akışı + bağımlılıklar

Opsiyonel seçenekler / varyantlar

Sonra tüm katmanları birleştirip AI botun anlayabileceği bir sıra ve mantık tablosu hazırlar

---

💡 Özetle: Önce meta-plan → sonra her dalın kendi planı → sonra tüm ağacı birleştiren plan.

İstersen ben bunu direkt AI botun kullanabileceği tam dallı tablo hâline getirip alt alt dallara kadar açabilirim.

Bunu yapayım mı, yoksa önce her dalın planını ayrı ayrı tek tek yazalım mı?

Tamam, önce yazıyorum.

Uygulama mimarisi planlama sırası

1) Önce uygulama tipi seçilir

Burada ilk soru şu:

Web mi?

Mobil mi?

Desktop mı?

API mi?

Mikroservis mi?

Gerçek zamanlı sistem mi?

Veri yoğun sistem mi?

Bu karar, bütün ağacın yönünü belirler. Çünkü her uygulama her dalı kullanmaz. Mesela CLI'da ekran/tasarım çok küçük kalır, ama API ve iş mantığı büyür. Mobilde offline, cache ve oturum çok önemlidir. E-ticarette ödeme, stok ve sipariş dalları büyür.

---

2) Sonra ürün kanalları planlanır

Her kanal ayrı ayrı düşünülür:

Web

Mobil

Desktop

CLI / Admin panel

Public API / Partner API

Bu aşamada şunlar netleşir:

Kullanıcı kim?

Nereden girecek?

Kaç farklı ekran olacak?

API dışarı açılacak mı?

Admin panel olacak mı?

---

3) Sunum katmanı planlanır

Bu katman kullanıcıya görünen kısımdır.

Plan sırası:

Ekranlar

Komponentler

State yönetimi

Routing

Formlar

Tema

Dil desteği

Erişilebilirlik

Hata / loading / boş durumlar

Oturum ekranları

Offline cache

Burada amaç şudur: kullanıcı ne görecek, neye basacak, ne nasıl davranacak?

---

4) Giriş katmanı / edge planlanır

Bu katman sistemin kapısıdır.

Plan sırası:

API Gateway var mı?

BFF gerekiyor mu?

Controller / handler / resolver nasıl olacak?

Request nasıl alınacak?

Response nasıl dönecek?

Auth nasıl yapılacak?

Rate limit gerekli mi?

Versioning olacak mı?

Pagination / filtering / sorting olacak mı?

CORS / CSRF korunacak mı?

Bu katman “kapıdan ne girer, ne çıkar” işini çözer.

---

5) Uygulama katmanı planlanır

Bu katman işleri sıraya koyan kısımdır.

Plan sırası:

Use case’ler

İş akışları

DTO / mapper

Transaction yönetimi

Idempotency

Retry / fallback

Saga / process manager

Uygulama servisleri

Burada sorulan soru şudur: “Kullanıcı bir şey yapınca sistem hangi adımlarla işlesin?”

---

6) Domain / iş katmanı planlanır

Bu katman uygulamanın asıl mantığıdır.

Plan sırası:

Entity

Value object

Aggregate

Domain service

Domain event

Policy / rule / specification

Factory

Bounded context

Domain invariants

Burada amaç kuralları yazmaktır.
Mesela:

Stok sıfırsa sipariş alınmaz.

Ödeme başarısızsa sipariş tamamlanmaz.

Kullanıcı onayı olmadan işlem yapılmaz.

---

7) Veri erişim katmanı planlanır

Bu katman veriyi çekip koyan kısımdır.

Plan sırası:

Repository

DAO

ORM

Query builder

Mapper / adapter

Migration

Seed / bootstrap data

Cache access

Search index access

File / blob access

Burada karar verilir:

Veri nereden gelecek?

Nasıl kaydedilecek?

Nasıl güncellenecek?

Cache kullanılacak mı?

Arama motoru lazım mı?

---

8) Entegrasyon / asenkron katman planlanır

Bu katman dış sistemlerle ve gecikmeli işler ile ilgilenir.

Plan sırası:

Message queue

Event bus

Pub/Sub

Background jobs / workers

Scheduler / cron

Webhook

External API client

Outbox / inbox pattern

Notification dispatch

Bu katman özellikle şunlarda şart olur:

Mail gönderme

Bildirim atma

Stok senkronu

Sipariş sonrası işlemler

Rapor üretme

Gece çalışan işler

---

9) Altyapı planlanır

Bu katman sistemin altında çalışan gerçek kaynaklardır.

Plan sırası:

Veritabanı

Cache sunucusu

Dosya / object storage

Search engine

Message broker

Secrets / config

Container / orchestration

CDN / DNS / load balancer

Network / firewall

Burada “hangi teknoloji nerede duracak” belli olur.

---

10) Kesişen endişeler planlanır

Bunlar her katmana yayılır.

Plan sırası:

Logging

Metrics

Tracing

Monitoring / alerting

Error handling

Security

Audit trail

Encryption

Feature flags

Observability

Backup / restore

Bunlar yoksa sistem çalışsa bile yönetilemez.

---

11) Yaşam döngüsü / operasyon planlanır

Kod yazmak kadar çalıştırmak da önemlidir.

Plan sırası:

Unit test

Integration test

E2E test

Contract test

CI/CD

Release / rollout

Rollback

Incident management

Maintenance

Bu aşama “bu sistem nasıl sağ kalacak?” sorusuna cevap verir.

---

12) Mimari stil planlanır

En son üst yapı seçilir.

Plan sırası:

Layered / N-Tier

Clean Architecture

Onion Architecture

Hexagonal Architecture

Modular Monolith

Microservices

Event-Driven

Serverless

CQRS / Event Sourcing

Bu seçim, bütün önceki katmanların nasıl yerleşeceğini belirler.

Kısa plan sırası

1. Uygulama tipi

2. Kanal

3. Sunum

4. Edge

5. Application

6. Domain

7. Data access

8. Integration

9. Infra

10. Cross-cutting concerns

11. Test / operasyon

12. Mimari stil

Bu sırayla gidilirse bot, kafasına göre zıplamaz; önce iskeleti kurar, sonra dalları açar.

Hatırlatma için zamanı yaz, onu da kurayım.

Kısacası: animasyon var diye iyi olmaz; iş görüyorsa iyidir. Material Design transitions'ı ekranlar arasında bağ kuran, kullanıcıya mekânsal süreklilik veren şeyler olarak tanımlar; Apple da hareketin durum, geri bildirim ve yönlendirme verdiğini söyler. Ama Apple ayrıca "Reduce Motion" desteğinin önemli olduğunu, çünkü bazı hareketlerin baş dönmesi, bulantı ve dikkat dağınıklığı yaratabildiğini belirtir. Yani iyi animasyon: yükleniyor, geçiş oluyor, başarı/hata oldu, ekran değişti gibi yerlerde; kötü animasyon: sırf süs diye her şeyi oynatmak.

API tarafında en mantıklı yol, tek bir sağlayıcıya gömülmek değil, sağlayıcıyı soyutlamak. Çünkü güncel API'ler farklı güçlü yönler sunuyor: OpenAI'nin Responses API'si durum tutan etkileşimleri, yerleşik araçları ve function calling'i destekliyor; Gemini API function calling ile modeli dış araçlara bağlamayı açıkça anlatıyor; Anthropic dokümanları da API anahtarı için environment variable kullanımını ve hatta OpenAI uyumlu uç noktayı desteklediğini söylüyor. Bu yüzden "hangi API daha iyi?" sorusunun doğru cevabı çoğu zaman "işe göre değişir" oluyor.

Senin bot için en doğru yapı şu: her API için ayrı bir kart olsun ve içinde provider, model, api key, base url, timeout, retry, enable/disable, test connection, fallback order alanları bulunsun. Böylece kullanıcı sadece anahtarı yapıştırıp aktif eder; ama sistem bunu tarayıcıda değil sunucu tarafında saklar. Google'ın dokümanı API anahtarını environment variable olarak vermeyi öneriyor ve hardcode etmeyi yalnızca geçici test için güvenli olmayan bir yöntem olarak yazıyor; ayrıca gizli bilgilerin istemci tarayıcıya açılmaması gerektiğini söylüyor. Stripe da sunucu tarafı kimlik doğrulamada secret key gerektiğini ve POST isteklerinde idempotency key kullanılmasını öneriyor.

Özellikle para, sipariş, kayıt, abonelik gibi işlerde ayrıca webhook ve idempotency koymak çok mantıklı. Stripe dokümanları, ağ hatasında aynı işlemin iki kez yapılmaması için idempotency key kullanmayı ve işlem sonucunu webhook ile takip etmeyi öneriyor. Bu, botun "gitti mi gitmedi mi" saçmalığını ciddi biçimde azaltır.

Benim önerim net:
1) Animasyonları sadece anlam taşıyan yerlere koy.
2) API'leri plugin gibi kur.
3) Anahtarı yapıştır-aktif et mantığını sadece admin panelde yap, ama saklamayı sunucuda yap.
4) LLM tarafında function calling + streaming + structured output destekleyen sağlayıcıları tercih et.

İstersen bunu bir sonraki mesajda sana "AI bot ayar paneli" tablo halinde çıkarayım: hangi kutu, hangi alan, hangi default değer, hangi zorunlu/opsiyonel.

```
UYGULAMA MİMARİSİ
├─ 0) ÜRÜN / KANAL
│  ├─ Web
│  │  ├─ Kurumsal web
│  │  ├─ SaaS paneli
│  │  ├─ E-ticaret
│  │  ├─ Blog / içerik
│  │  └─ Portal / intranet
│  ├─ Mobil
│  │  ├─ Native iOS
│  │  ├─ Native Android
│  │  ├─ Flutter / React Native
│  │  ├─ Offline-first
│  │  └─ Super-app modülü
│  ├─ Desktop
│  │  ├─ Native masaüstü
│  │  ├─ Electron
│  │  └─ Tauri
│  ├─ CLI / Admin panel
│  │  ├─ Komut satırı aracı
│  │  ├─ Admin dashboard
│  │  └─ DevOps aracı
```

```
│   ├─ Public API / Partner API
│   │   ├─ REST
│   │   ├─ GraphQL
│   │   └─ gRPC
│   └─ Gerçek zamanlı sistem
│       ├─ Chat
│       ├─ Canlı skor
│       ├─ Trading
│       └─ Game backend
│
├─ 1) İSTEMCİ / SUNUM KATMANI
│   ├─ Ekranlar / Sayfalar
│   │   ├─ Ana sayfa
│   │   ├─ Liste
│   │   ├─ Detay
│   │   ├─ Form
│   │   ├─ Profil
│   │   └─ Admin ekranları
│   ├─ Komponentler / Widget'lar
│   │   ├─ Button
│   │   ├─ Input
│   │   ├─ Modal
│   │   ├─ Card
│   │   ├─ Table
│   │   └─ Chart
│   ├─ State management
│   │   ├─ Local state
│   │   ├─ Global state
│   │   ├─ Server state
│   │   ├─ Session state
│   │   └─ Cache state
│   ├─ Routing / Navigation
│   │   ├─ Route guard
│   │   ├─ Nested route
│   │   ├─ Deep link
│   │   └─ Breadcrumb
│   ├─ Formlar / Input validation
│   │   ├─ Client validation
│   │   ├─ Server validation
│   │   ├─ Mask / format
│   │   └─ Error message
│   ├─ Tema / Stil / UI kit
│   │   ├─ Design system
│   │   ├─ Component library
│   │   ├─ Theme switcher
│   │   ├─ Responsive layout
│   │   └─ Design tokens
│   ├─ Dil desteği / i18n
```

```
│   │   ├─ Locale detection
│   │   ├─ Translation files
│   │   ├─ Pluralization
│   │   └─ Number/date format
│   ├─ Erişilebilirlik / accessibility
│   │   ├─ Keyboard navigation
│   │   ├─ Screen reader labels
│   │   ├─ Focus management
│   │   └─ Contrast
│   ├─ Hata / empty / loading state
│   │   ├─ Skeleton
│   │   ├─ Spinner
│   │   ├─ Retry ekranı
│   │   ├─ Boş veri ekranı
│   │   └─ Offline ekranı
│   ├─ Oturum ekranları
│   │   ├─ Login
│   │   ├─ Logout
│   │   ├─ Refresh token
│   │   ├─ Forgot password
│   │   └─ 2FA / MFA
│   └─ Offline cache / local storage / analytics
│       ├─ localStorage
│       ├─ sessionStorage
│       ├─ IndexedDB
│       ├─ crash analytics
│       └─ usage analytics
│
├─ 2) GİRİŞ KATMANI / EDGE
│   ├─ API Gateway
│   │   ├─ Routing
│   │   ├─ Auth validation
│   │   ├─ Rate limiting
│   │   ├─ Transform
│   │   └─ Logging
│   ├─ BFF (Backend for Frontend)
│   │   ├─ Web BFF
│   │   ├─ Mobile BFF
│   │   ├─ Admin BFF
│   │   └─ Aggregation endpoint
│   ├─ Controller / Handler / Resolver
│   │   ├─ REST controller
│   │   ├─ GraphQL resolver
│   │   ├─ gRPC handler
│   │   └─ CLI command handler
│   ├─ Request parsing / validation
│   │   ├─ Schema validation
│   │   ├─ DTO binding
```

```
│   │   └─ Sanitization
│   ├─ Response shaping / formatting
│   │   ├─ DTO response
│   │   ├─ Pagination payload
│   │   └─ Error payload
│   ├─ AuthN / AuthZ
│   │   ├─ Session auth
│   │   ├─ JWT
│   │   ├─ OAuth2 / OpenID Connect
│   │   └─ RBAC / ABAC
│   ├─ Rate limit / throttle
│   ├─ Versioning
│   ├─ Pagination / filtering / sorting
│   └─ CORS / CSRF / anti-abuse
│
├─ 3) UYGULAMA KATMANI
│   ├─ Use case'ler
│   │   ├─ Create
│   │   ├─ Update
│   │   ├─ Delete
│   │   ├─ Approve
│   │   ├─ Cancel
│   │   └─ Publish
│   ├─ İş akışı orkestrasyonu
│   │   ├─ Step orchestration
│   │   ├─ Conditional flow
│   │   └─ Compensation flow
│   ├─ DTO / Mapper
│   │   ├─ Request DTO
│   │   ├─ Response DTO
│   │   ├─ Command / Query model
│   │   └─ Mapping rules
│   ├─ Transaction yönetimi
│   │   ├─ Atomic transaction
│   │   ├─ Isolation level
│   │   └─ Commit / rollback
│   ├─ Idempotency
│   │   ├─ Idempotency key
│   │   ├─ Duplicate prevention
│   │   └─ Replay protection
│   ├─ Retry / fallback
│   │   ├─ Retry policy
│   │   ├─ Circuit breaker
│   │   └─ Fallback response
│   ├─ Saga / process manager
│   │   ├─ Orchestrated saga
│   │   ├─ Choreographed saga
│   │   └─ Compensation action
```

```
│   └─ Uygulama servisleri
│      ├─ Order service
│      ├─ Payment service
│      ├─ Notification service
│      └─ Search service
│
├─ 4) DOMAIN / İŞ KATMANI
│   ├─ Entity
│   │   ├─ Identity
│   │   ├─ Lifecycle
│   │   └─ State transitions
│   ├─ Value Object
│   │   ├─ Immutable data
│   │   ├─ Validation
│   │   └─ Equality by value
│   ├─ Aggregate
│   │   ├─ Aggregate root
│   │   ├─ Consistency boundary
│   │   └─ Child entities
│   ├─ Domain Service
│   │   ├─ Business calculation
│   │   ├─ Cross-entity rule
│   │   └─ Policy enforcement
│   ├─ Domain Event
│   │   ├─ Event generation
│   │   ├─ Event reaction
│   │   └─ Integration event
│   ├─ Policy / Rule / Specification
│   │   ├─ Rule engine
│   │   ├─ Invariant
│   │   └─ Eligibility check
│   ├─ Factory
│   │   ├─ Object creation
│   │   ├─ Complex construction
│   │   └─ Domain-safe creation
│   ├─ Bounded Context
│   └─ Domain invariants
│
├─ 5) VERİ ERİŞİM KATMANI
│   ├─ Repository
│   │   ├─ Read repository
│   │   ├─ Write repository
│   │   └─ Custom queries
│   ├─ DAO
│   ├─ ORM
│   │   ├─ Entity mapping
│   │   ├─ Lazy / eager load
│   │   └─ Change tracking
```

```
│   ├─ Query builder
│   │   ├─ Dynamic filter
│   │   ├─ Join
│   │   └─ Sorting
│   ├─ Mapper / adapter
│   ├─ Migration
│   ├─ Seed / bootstrap data
│   ├─ Cache access
│   │   ├─ Read-through
│   │   ├─ Write-through
│   │   └─ Cache invalidation
│   ├─ Search index access
│   └─ File / blob access
│
├─ 6) ENTEGRASYON / ASENKRON KATMAN
│   ├─ Message queue
│   │   ├─ Kafka
│   │   ├─ RabbitMQ
│   │   ├─ SQS
│   │   └─ Dead letter queue
│   ├─ Event bus
│   ├─ Pub/Sub
│   ├─ Background jobs / workers
│   │   ├─ Queue worker
│   │   ├─ Batch worker
│   │   └─ Scheduled worker
│   ├─ Scheduler / cron
│   │   ├─ Periodic sync
│   │   ├─ Cleanup job
│   │   └─ Report job
│   ├─ Webhook
│   ├─ External API client
│   ├─ Outbox / inbox pattern
│   └─ Notification dispatch
│
├─ 7) ALTYAPI / INFRA
│   ├─ Veritabanı
│   │   ├─ SQL
│   │   │   ├─ Postgres
│   │   │   ├─ MySQL
│   │   │   └─ MSSQL
│   │   ├─ NoSQL
│   │   │   ├─ MongoDB
│   │   │   └─ DynamoDB
│   │   ├─ Graph DB
│   │   └─ Time-series DB
│   ├─ Cache sunucusu
│   │   ├─ Redis
```

```
│   │   └─ Memcached
│   ├─ Dosya / object storage
│   │   ├─ S3
│   │   ├─ Azure Blob
│   │   ├─ GCS
│   │   └─ File system
│   ├─ Search engine
│   │   ├─ Elasticsearch
│   │   └─ OpenSearch
│   ├─ Message broker
│   ├─ Secrets / config
│   ├─ Container / orchestration
│   │   ├─ Docker
│   │   ├─ Kubernetes
│   │   └─ Helm / manifests
│   ├─ CDN / DNS / load balancer
│   └─ Network / firewall
│
├─ 8) KESİŞEN ENDİŞELER
│   ├─ Logging
│   │   ├─ Structured logs
│   │   └─ Correlation id
│   ├─ Metrics
│   ├─ Tracing
│   ├─ Monitoring / alerting
│   │   ├─ Dashboard
│   │   ├─ Alert rule
│   │   └─ On-call
│   ├─ Error handling
│   │   ├─ Exception mapping
│   │   ├─ Retryable error
│   │   └─ User-friendly message
│   ├─ Security
│   │   ├─ Auth
│   │   ├─ Authorization
│   │   ├─ Secrets
│   │   └─ Threat mitigation
│   ├─ Audit trail
│   ├─ Encryption
│   ├─ Feature flags
│   ├─ Observability
│   └─ Backup / restore
│
├─ 9) YAŞAM DÖNGÜSÜ / OPERASYON
│   ├─ Testler
│   │   ├─ Unit
│   │   ├─ Integration
│   │   ├─ E2E
```

```
│   │   ├─ Contract tests
│   │   └─ Smoke test
│   ├─ CI / CD
│   ├─ Release / rollout
│   ├─ Rollback
│   ├─ Incident management
│   └─ Maintenance
│
├─ 10) AI / OTOMASYON
│   ├─ Model provider katmanı
│   │   ├─ OpenAI
│   │   ├─ Gemini
│   │   ├─ Anthropic
│   │   └─ Local model
│   ├─ Provider adapter
│   │   ├─ Unified request format
│   │   ├─ Unified response format
│   │   ├─ Streaming
│   │   ├─ Tool/function calling
│   │   └─ Structured output
│   ├─ API key management
│   │   ├─ Secret vault
│   │   ├─ Env var
│   │   ├─ Per-user key
│   │   └─ Rotation
│   ├─ Model registry
│   │   ├─ Model name
│   │   ├─ Temperature
│   │   ├─ Max tokens
│   │   ├─ Timeout
│   │   └─ Fallback model
│   ├─ Prompt templates
│   ├─ Tool registry
│   ├─ Webhook / callback
│   ├─ Usage metering
│   └─ Safety / moderation
│
└─ 11) MİMARİ STİL / YAPI TÜRÜ
    ├─ Layered / N-Tier
    │   ├─ Sunum
    │   ├─ Uygulama
    │   ├─ Domain
    │   ├─ Veri erişim
    │   └─ Veri
    ├─ Clean Architecture
    │   ├─ Entities
    │   ├─ Use Cases
    │   ├─ Interface Adapters
```

```
│   └─ Frameworks & Drivers
├─ Onion Architecture
│   ├─ Core
│   ├─ Application
│   ├─ Infrastructure
│   └─ External systems
├─ Hexagonal Architecture
│   ├─ Domain
│   ├─ Ports
│   ├─ Adapters
│   └─ Outside world
├─ Modular Monolith
│   ├─ Module A
│   ├─ Module B
│   ├─ Module C
│   ├─ Shared kernel
│   └─ Internal events
├─ Microservices
│   ├─ Service A
│   ├─ Service B
│   ├─ Service C
│   ├─ API Gateway
│   ├─ Event / Message Bus
│   └─ Service discovery
├─ Event-Driven
│   ├─ Producer
│   ├─ Consumer
│   ├─ Event broker
│   ├─ Event store
│   └─ Projection
├─ Serverless
│   ├─ Functions
│   ├─ Event triggers
│   ├─ Managed services
│   └─ Cold start
└─ CQRS / Event Sourcing
    ├─ Command model
    ├─ Query model
    ├─ Event store
    ├─ Projection / read model
    └─ Snapshot


ÜRÜN / KANAL
├─ Kullanıcı uygulaması
│   ├─ Web
│   ├─ Mobil
│   └─ Desktop
```

```
├─ Yönetim paneli
│   ├─ Admin panel
│   ├─ Moderasyon paneli
│   ├─ Operasyon paneli
│   ├─ Destek paneli
│   └─ Rapor / analitik paneli
└─ API katmanı
    ├─ Public API
    ├─ Partner API
    └─ Internal API
```

```
ADMIN PANELİ
├─ Kullanıcı Yönetimi
│   ├─ Kullanıcı listesi / filtreleme
│   ├─ Rol & Yetki atama
│   ├─ Kullanıcı oluştur / sil / düzenle
│   ├─ Aktivite geçmişi / loglar
│   └─ Hesap durum yönetimi (aktif, pasif, askıya alma)
│
├─ İçerik / Ürün Yönetimi
│   ├─ Ürün ekle / sil / düzenle
│   ├─ Kategori yönetimi
│   ├─ Medya yönetimi (resim, video)
│   ├─ Yayınlama / taslak yönetimi
│   └─ İçerik onay süreci / moderasyon
│
├─ Sipariş / İşlem Yönetimi
│   ├─ Sipariş listesi / filtreleme
│   ├─ Sipariş durumu yönetimi
│   ├─ Ödeme & fatura kontrolü
│   ├─ İade / değişim işlemleri
│   └─ Satış raporları
│
├─ Destek & Operasyon
│   ├─ Destek talepleri yönetimi
│   ├─ Canlı chat / mesaj yönetimi
│   ├─ Sistem uyarıları / bildirimler
│   └─ Görev / ticket atama
│
├─ Raporlama & Analitik
│   ├─ Kullanıcı davranış analizi
│   ├─ Satış / gelir raporları
│   ├─ Performans metriği (hız, uptime)
│   ├─ Hata & log analizi
│   └─ KPI / hedef takibi
│
├─ Sistem & Ayarlar
```

```
│   ├─ Genel uygulama ayarları
│   ├─ Feature flags yönetimi
│   ├─ API key yönetimi
│   ├─ Backup / restore işlemleri
│   └─ Güvenlik ayarları (2FA, IP kısıtlama, audit log)
│
└─ Entegrasyon & API
   ├─ Harici servis bağlantıları
   │   ├─ Ödeme servisleri
   │   ├─ SMS / e-posta servisleri
   │   └─ Analitik servisleri (GA, Mixpanel vb.)
   ├─ İç API yönetimi
   │   ├─ Endpoint aktivasyonu / pasifleştirme
   │   └─ API kullanım logları
   └─ Webhook / event yönetimi

ADMIN PANELİ – DETAYLI PLAN
├─ Kullanıcı Yönetimi
│   ├─ Kullanıcı listesi / filtreleme
│   │   ├─ Animasyon: sonsuz scroll, hızlı filtreleme animasyonu
│   │   ├─ Geçiş: hover highlight, detay açılır pencereler
│   │   ├─ İnovasyon: canlı arama + AI öneri (ör. spam kullanıcı uyarısı)
│   │   └─ API: REST / GraphQL, hızlı filtre için ElasticSearch entegrasyonu
│   ├─ Rol & Yetki atama
│   │   ├─ Animasyon: sürükle-bırak yetki atama
│   │   ├─ Geçiş: dropdown ile role hover preview
│   │   ├─ İnovasyon: AI rol önerisi (kullanıcının davranışına göre)
│   │   └─ API: Auth0 / Keycloak / Firebase Auth
│   ├─ Kullanıcı oluştur / sil / düzenle
│   │   ├─ Animasyon: modal açılış/kapanış, onay geçiş animasyonu
│   │   ├─ İnovasyon: form doldurma önerileri / validation hints
│   │   └─ API: REST CRUD, gRPC opsiyonel
│   ├─ Aktivite geçmişi / loglar
│   │   ├─ Animasyon: zaman çizelgesi kaydırma animasyonu
│   │   ├─ Geçiş: detay açılır paneller
│   │   └─ API: ELK Stack, Splunk veya Datadog
│   └─ Hesap durum yönetimi
│       ├─ Animasyon: durum değişim toggle animasyonu
│       ├─ İnovasyon: otomatik risk tespiti (AI ile)
│       └─ API: Auth provider + internal user service

├─ İçerik / Ürün Yönetimi
│   ├─ Ürün ekle / sil / düzenle
│   │   ├─ Animasyon: görsel preview, drag-drop upload
│   │   ├─ İnovasyon: AI ürün açıklaması önerisi
│   │   └─ API: REST / GraphQL + S3 / Blob storage
│   ├─ Kategori yönetimi
│   │   ├─ Animasyon: kategori ağacı expand/collapse
```

```
│  │   ├─ İnovasyon: otomatik kategori önerisi
│  │   └─ API: ElasticSearch / DB queries
│  ├─ Medya yönetimi
│  │   ├─ Animasyon: thumbnail hover zoom
│  │   ├─ İnovasyon: AI image tagging
│  │   └─ API: Cloudinary / AWS S3
│  ├─ Yayınlama / taslak yönetimi
│  │   ├─ Animasyon: timeline slider
│  │   └─ API: internal CMS service
│  └─ İçerik onay / moderasyon
│      ├─ Animasyon: approve/reject swipe
│      ├─ İnovasyon: AI moderation önerisi
│      └─ API: Perspective API / internal rules engine

├─ Sipariş / İşlem Yönetimi
│  ├─ Sipariş listesi / filtreleme
│  │   ├─ Animasyon: infinite scroll, hover highlight
│  │   └─ API: REST + ElasticSearch
│  ├─ Sipariş durumu yönetimi
│  │   ├─ Animasyon: durum change animation
│  │   └─ API: internal order service
│  ├─ Ödeme & fatura kontrolü
│  │   ├─ Animasyon: modal fade-in
│  │   └─ API: Stripe / PayPal / Braintree
│  ├─ İade / değişim işlemleri
│  │   ├─ Animasyon: step-by-step progress
│  │   └─ API: order service + payment provider
│  └─ Satış raporları
│      ├─ Animasyon: chart transition
│      ├─ İnovasyon: AI sales trend prediction
│      └─ API: BI tool integration (Tableau, PowerBI)

├─ Destek & Operasyon
│  ├─ Destek talepleri yönetimi
│  │   ├─ Animasyon: ticket expand/collapse
│  │   └─ API: Zendesk / Freshdesk
│  ├─ Canlı chat / mesaj yönetimi
│  │   ├─ Animasyon: message bubble animations
│  │   └─ API: Twilio / Pusher / WebSocket
│  ├─ Sistem uyarıları / bildirimler
│  │   ├─ Animasyon: toast / snackbar
│  │   └─ API: internal notification service
│  └─ Görev / ticket atama
│      ├─ Animasyon: drag-drop assignment
│      └─ API: internal task service

├─ Raporlama & Analitik
│  ├─ Kullanıcı davranış analizi
```

```
│   │   └─ API: GA4 / Mixpanel / internal analytics
│   ├─ Satış / gelir raporları
│   │   └─ API: internal order + BI tool
│   ├─ Performans metriği
│   │   └─ API: Prometheus / Grafana
│   ├─ Hata & log analizi
│   │   └─ API: ELK Stack / Datadog
│   └─ KPI / hedef takibi
│       └─ API: internal metrics service

├─ Sistem & Ayarlar
│   ├─ Genel uygulama ayarları
│   ├─ Feature flags yönetimi
│   │   └─ API: LaunchDarkly / internal flags
│   ├─ API key yönetimi
│   ├─ Backup / restore işlemleri
│   │   └─ API: internal backup service + cloud storage
│   └─ Güvenlik ayarları
│       └─ API: Auth provider + internal security service

└─ Entegrasyon & API
    ├─ Harici servis bağlantıları
    │   ├─ Ödeme servisleri
    │   ├─ SMS / e-posta servisleri
    │   └─ Analitik servisleri
    ├─ İç API yönetimi
    │   ├─ Endpoint aktivasyonu / pasifleştirme
    │   └─ API kullanım logları
    └─ Webhook / event yönetimi
        └─ API: internal event service
```

Tamam, bunu hazır admin panel şablonu gibi düşünebiliriz. Aşağıdaki yapı, botunun panel üretirken sırayı bozmasını engeller.

Admin panel için tasarım iskeleti

1) Genel kabuk

Bu her admin panelde olur:

Sol menü: bölümler

Üst bar: arama, bildirim, profil, çıkış

Orta alan: liste, form, detay

Sağ panel: hızlı özet, filtre, yardım, son işlemler

Alt durum alanı: yükleniyor, hata, başarı mesajı

2) Sayfa akışı

Her modül şu sırayla kurulmalı:

Liste ekranı

Filtreleme

Detay ekranı

Düzenleme formu

Onay / silme modalı

Log / geçmiş ekranı

Rapor / özet ekranı

3) Animasyon ve geçiş mantığı

Animasyonlar gösteriş için değil, anlam için olmalı:

Sayfa geçişi: hafif fade + slide

Modal açılışı: kısa fade in

Modal kapanışı: fade out

Liste yenileme: skeleton veya shimmer

Filtre uygulama: küçük loading state

Kart açma / kapama: collapse animasyonu

Bildirim: toast / snackbar

Durum değişimi: renk ve ikon değişimi

Tab geçişi: underline slide

Chart / rapor: yumuşak çizim animasyonu

4) İnovasyon nerede gerçekten işe yarar

Bunlar saçma değil, doğru yerde kullanılırsa işe yarar:

AI destekli arama: kullanıcıyı hızlı bulmak için

Akıllı öneri: rol, kategori, etiket önerme

Toplu işlem: çok kayıt üzerinde tek işlem

Risk uyarısı: şüpheli kullanıcı / işlem işaretleme

Otomatik kategorileme: içerik ya da ürün sınıflandırma

Canlı özet: rapor ekranında otomatik çıkarım

5) API mantığı

Her modül için API'yi şöyle ayırmak mantıklı:

Listeleme → GET

Detay → GET /id

Ekleme → POST

Güncelleme → PUT veya PATCH

Silme → DELETE

Toplu işlem → POST /bulk

Arama / filtre → GET + query params

Log / geçmiş → ayrı log endpoint'i

Rapor → ayrı analytics endpoint'i

6) "Key yapıştır aktif et" yapısı doğru mu?

Evet, ama doğru yerde.

En iyi yapı şu:

Kullanıcı panelde servis seçer

API key girer

Test bağlantısı yapılır

Başarılıysa aktif edilir

Key sunucu tarafında güvenli saklanır

İstemciye açık verilmez

Yani panelde kolay kurulum olur, arkada güvenli saklama olur.

Admin panel modülleri için örnek hazır düzen

Modül	Ekranlar	Animasyon	İnovasyon	API
Kullanıcı Yönetimi	Liste, detay, rol atama	Satır açılma, modal	Risk uyarısı, akıllı arama	REST + arama servisi
İçerik / Ürün	Liste, form, medya	Görsel preview, drag-drop	Otomatik etiketleme	REST + file storage
Sipariş / İşlem	Liste, durum, iade	Durum geçişi	Sahte işlem tespiti	Order service
Destek	Ticket, sohbet, görev	Panel açılma, toast	Öncelik önerisi	Ticket API
Raporlama	Grafik, KPI, özet	Chart animasyonu	Trend tahmini	Analytics API
Ayarlar	Genel, güvenlik, flags	Toggle animasyonu	Akıllı öneri	Config API

Botun için en doğru genel kural

Bot panel üretirken şu sırayı izlesin:

1. Kanalı seçsin

2. Admin panel var mı karar versin

3. Menüleri çıkarsın

4. Her menü için liste/detay/form/log kurulsun

5. API uçlarını bağlasın

6. Animasyonları eklesin

7. Güvenlik ve yetkiyi koysun

8. Log, rapor, export, backup eklesin

Bu sıra bozulursa panel dağılır, sonra eline yüzüne bulaşır.

İstersen bir sonraki adımda bunu sana “admin panel üretim promptu” gibi tek parça, doğrudan AI botuna yapıştırılacak formatta verebilirim.

Üretilecek admin panel için tüm yapı aşağıdaki kurallara göre oluşturulsun:

1) Kanal seçimi: Web / Mobil / Desktop / CLI / Partner API. Panel buna göre adaptif olsun.

2) Menü ve modüller:
- Kullanıcı Yönetimi
- İçerik / Ürün Yönetimi
- Sipariş / İşlem Yönetimi
- Destek & Operasyon
- Raporlama & Analitik
- Sistem & Ayarlar
- Entegrasyon & API

3) Her modül için ekran akışı:
- Liste ekranı
- Filtreleme / arama
- Detay ekranı
- Form (ekle / düzenle / sil)
- Onay modalı
- Log / geçmiş ekranı
- Rapor / özet ekranı

4) Animasyon ve geçişler:
- Sayfa geçişi: fade + slide
- Modal açılışı / kapanışı: fade in/out
- Liste yenileme: skeleton / shimmer
- Filtre uygulama: loading state
- Kart açma / kapama: collapse animasyonu
- Bildirim: toast / snackbar

- Durum değişimi: renk + ikon değişimi
- Tab geçişi: underline slide
- Chart / rapor: yumuşak çizim animasyonu

5) İnovasyon:
- AI destekli arama ve öneri (kullanıcı, rol, kategori, etiket)
- Toplu işlem
- Risk uyarısı / sahte işlem tespiti
- Otomatik kategorileme / etiketleme
- Canlı özet ve trend tahmini

6) API entegrasyonu:
- Listeleme → GET
- Detay → GET /id
- Ekleme → POST
- Güncelleme → PUT/PATCH
- Silme → DELETE
- Toplu işlem → POST /bulk
- Arama / filtre → GET + query params
- Log / geçmiş → log endpoint
- Rapor → analytics endpoint
- Key yönetimi: kullanıcı key girer, test edilir, aktif edilir, güvenli saklanır, istemciye açık verilmez

7) Modül özel API önerileri:
- Kullanıcı: Auth0 / Keycloak / Firebase Auth + ElasticSearch
- İçerik / Ürün: REST + Cloud Storage / S3 + AI etiketleme
- Sipariş: Order service + Stripe / PayPal
- Destek: Zendesk / Freshdesk + Twilio / Pusher
- Raporlama: GA4 / Mixpanel / internal analytics + BI tool
- Ayarlar: internal config / LaunchDarkly
- Entegrasyon: harici API'ler (ödeme, SMS, e-posta, webhook)

8) Panelin genel kabuk yapısı:
- Sol menü: modüller
- Üst bar: arama, bildirim, profil, çıkış
- Orta alan: liste, form, detay
- Sağ panel: hızlı özet, filtre, yardım, son işlemler
- Alt durum alanı: yükleniyor, hata, başarı mesajı

9) Bot panel üretirken izlenecek sıra:
1. Kanal seçimi
2. Admin panel var mı karar
3. Menüleri oluştur
4. Her menü için liste/detay/form/log kur
5. API uçlarını bağla
6. Animasyonları ekle
7. Güvenlik / yetki ekle

8. Log, rapor, export, backup ekle

10) Sonuç: Tamamen çalışır, modern, güvenli, AI destekli admin panel şablonu.


```
{
  "channel": ["Web", "Mobil", "Desktop", "CLI", "Partner API"],
  "adminPanel": {
    "shell": {
      "leftMenu": ["Kullanıcı Yönetimi", "İçerik/Ürün Yönetimi", "Sipariş/İşlem Yönetimi",
"Destek & Operasyon", "Raporlama & Analitik", "Sistem & Ayarlar", "Entegrasyon & API"],
      "topBar": ["Arama", "Bildirim", "Profil", "Çıkış"],
      "mainArea": ["Liste", "Form", "Detay"],
      "rightPanel": ["Hızlı Özet", "Filtre", "Yardım", "Son İşlemler"],
      "bottomStatus": ["Yükleniyor", "Hata", "Başarı Mesajı"]
    },
    "modules": {
      "Kullanıcı Yönetimi": {
        "screens": ["Liste", "Filtreleme/Arama", "Detay", "Form Ekle/Düzenle/Sil", "Onay
Modalı", "Log/Günlük", "Rapor/Özet"],
        "animations": ["Satır Açılma", "Modal Fade In/Out", "Toast/Snackbar", "Durum
Renk+İkon"],
        "innovation": ["AI Destekli Arama", "Risk Uyarısı", "Toplu İşlem"],
        "api": ["GET Liste", "GET /id Detay", "POST Ekle", "PUT/PATCH Güncelle", "DELETE
Sil", "POST /bulk Toplu İşlem"]
      },
      "İçerik/Ürün Yönetimi": {
        "screens": ["Liste", "Filtreleme", "Detay", "Form", "Onay Modalı", "Log", "Rapor"],
        "animations": ["Preview Görsel", "Drag&Drop", "Skeleton", "Collapse Kart"],
        "innovation": ["Otomatik Etiketleme", "AI Öneri", "Toplu İşlem"],
        "api": ["REST CRUD", "File Storage / S3", "AI Tagging Endpoint"]
      },
      "Sipariş/İşlem Yönetimi": {
        "screens": ["Liste", "Detay", "Durum Değişimi", "Form", "Onay Modalı", "Log", "Rapor"],
        "animations": ["Durum Geçişi", "Toast Bildirim", "Collapse Kart"],
        "innovation": ["Sahte İşlem Tespiti", "Toplu İşlem"],
        "api": ["Order Service", "Payment Gateway (Stripe/PayPal)"]
      },
      "Destek & Operasyon": {
        "screens": ["Ticket Liste", "Detay", "Görev Atama", "Form", "Onay Modalı", "Log",
"Rapor"],
        "animations": ["Modal Fade", "Toast", "Collapse"],
        "innovation": ["Öncelik Önerisi", "AI Chatbot"],
        "api": ["Ticket API", "Sohbet / Twilio / Pusher"]
      },
      "Raporlama & Analitik": {
        "screens": ["Grafik", "KPI", "Özet", "Tablo", "Export"],
        "animations": ["Chart Animasyonu", "Fade", "Slide"],
        "innovation": ["Trend Tahmini", "Canlı Özet"],
        "api": ["Analytics API (GA4, Mixpanel)", "Internal BI"]
```

```
      },
      "Sistem & Ayarlar": {
        "screens": ["Genel", "Güvenlik", "Feature Flags", "Backup/Restore"],
        "animations": ["Toggle Animasyonu", "Fade", "Slide"],
        "innovation": ["AI Öneri", "Rol Bazlı Yetki"],
        "api": ["Config API", "LaunchDarkly"]
      },
      "Entegrasyon & API": {
        "screens": ["Liste", "Form Ekle/Düzenle", "Test Bağlantı", "Log", "Rapor"],
        "animations": ["Fade", "Slide", "Toast"],
        "innovation": ["Key Aktifleştirme", "Webhook Test"],
        "api": ["External API Clients", "Webhook", "Pub/Sub", "Message Queue"]
      }
    },
    "workflow": [
      "Kanal Seçimi",
      "Admin Panel Var mı Karar",
      "Menüleri Oluştur",
      "Liste/Detay/Form/Log Kur",
      "API Uçlarını Bağla",
      "Animasyonları Ekleyin",
      "Güvenlik / Yetki Ekleyin",
      "Log, Rapor, Export, Backup Ekleyin"
    ]
  }
}


{
  "channel": ["Web", "Mobil", "Desktop"],
  "adminPanel": {
    "menus": ["Kullanıcılar", "Ürünler", "Siparişler", "Destek", "Raporlar", "Ayarlar"],
    "screens": {
      "Liste": ["Filtreleme", "Arama", "Detay"],
      "Form": ["Ekle", "Düzenle", "Sil"],
      "Rapor": ["Grafik", "Tablo", "Export"]
    },
    "animations": ["Fade", "Slide", "Toast", "Collapse"],
    "api": ["GET Liste", "GET /id", "POST Ekle", "PUT/PATCH Güncelle", "DELETE Sil"],
    "workflow": ["Kanal Seçimi", "Menü Oluştur", "Ekranları Bağla", "API Uçlarını Bağla",
"Animasyonları Ekle"]
  }
}
```